

BÁO CÁO CHI TIẾT & MÃ NGUỒN

MODULE 1: ĐĂNG NHẬP & THIẾT LẬP THỂ TRẠNG

Dự án: Ứng dụng Quản lý Dinh dưỡng Cá nhân với AI (Smart Nutrition)

Nền tảng: Android Native (Kotlin) + Firebase Auth & Firestore + Jetpack Compose

Người thực hiện: Thành viên 1 - Module Auth & Profile Setup

Thời gian báo cáo: Tháng 7 năm 2026

BÁO CÁO CHI TIẾT MODULE 1: ĐĂNG NHẬP & THIẾT LẬP THẺ TRẠNG

Dự án: Ứng dụng Quản lý Dinh dưỡng Cá nhân với AI (Smart Nutrition)

MỤC TIÊU CỐT LÕI:

Vai trò của Module 1 trong dự án: Đây là module nền tảng **bắt buộc phải hoàn thành đầu tiên**. Nó quản lý định danh người dùng (`UID`) từ Firebase Auth và cung cấp các chỉ số thể trạng cốt lõi (Calories Target, Macros) cho tất cả các module còn lại (Pantry, AI Meal Planner, Habit Tracker, Analytics) truy vấn và sử dụng.

GỢI Ý/TÀI LIỆU:

Thư mục lưu trữ mã nguồn riêng: Toàn bộ mã nguồn liên quan đến Module 1 đã được sao chép và đóng gói vào thư mục `module1_code` tại thư mục gốc của dự án. Bạn có thể sử dụng thư mục này để nén (zip) gửi nộp bài hoặc mở xem nhanh các file code.

1. TỔNG QUAN VÀ PHÂN CÔNG CỦA THÀNH VIÊN 1

- **Tên Module:** Auth & Profile Setup (Đăng nhập & Thiết lập thẻ trạng).
- **Độ khó:** ★★ (Dễ/Trung bình).
- **Thành viên phụ trách:** Thành viên 1.
- **Mục tiêu chính:**

1. Xác thực danh tính người dùng bảo mật và thuận tiện.
2. Thiết lập hồ sơ thẻ trạng cơ bản, từ đó tính toán tự động và chính xác mục tiêu năng lượng và dinh dưỡng hàng ngày (chạy local trên app).
3. Lưu trữ dữ liệu đồng bộ đám mây và hoạt động offline (Offline-first).
4. Cung cấp chức năng ghi chép cân nặng hàng ngày để theo dõi sự biến động (weight logging).

2. DANH SÁCH MÀN HÌNH (SCREENS) VÀ LƯỒNG ĐI (FLOW)

Module sử dụng **Jetpack Compose** kết hợp với **Material Design 3** đem lại giao diện hiện đại và mượt mà.

2.1 Danh sách màn hình UI

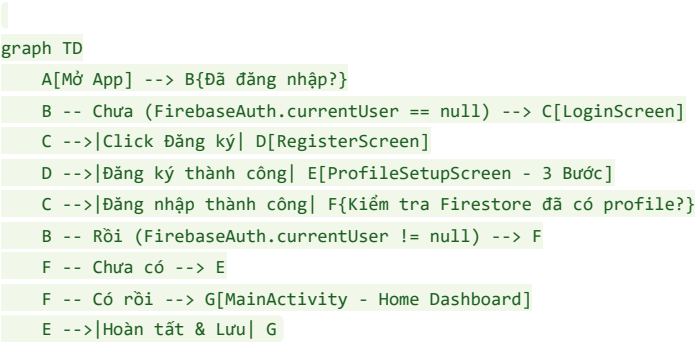
1. **LoginScreen:** Form đăng nhập bằng Email/Password + Nút đăng nhập nhanh bằng Google + Nút chuyển sang Đăng ký.
2. **RegisterScreen:** Form đăng ký (Email, Mật khẩu có xác thực độ mạnh, Xác nhận mật khẩu).
3. **ProfileSetupScreen:** Giao diện Wizard 3 bước (chỉ hiện 1 lần duy nhất sau khi đăng ký tài khoản mới):

- **Bước 1 (Thông tin cơ bản):** Tên hiển thị, Giới tính, Năm sinh (dùng Slider + Ô nhập).
- **Bước 2 (Chỉ số cơ thể):** Chiều cao, Cân nặng (Slider + Ô nhập) $\rightarrow$ Xem trước chỉ số BMI tức thời.
- **Bước 3 (Mục tiêu & Vận động):** Chọn mục tiêu (Giảm cân, Tăng cơ, Duy trì) + Mức độ vận động (5 cấp độ) $\rightarrow$ Hiển thị toàn bộ kết quả tính toán (BMI, BMR, TDEE, Target Calories, Macros).

4. **ProfileViewScreen:** Xem và cập nhật thông tin cá nhân.

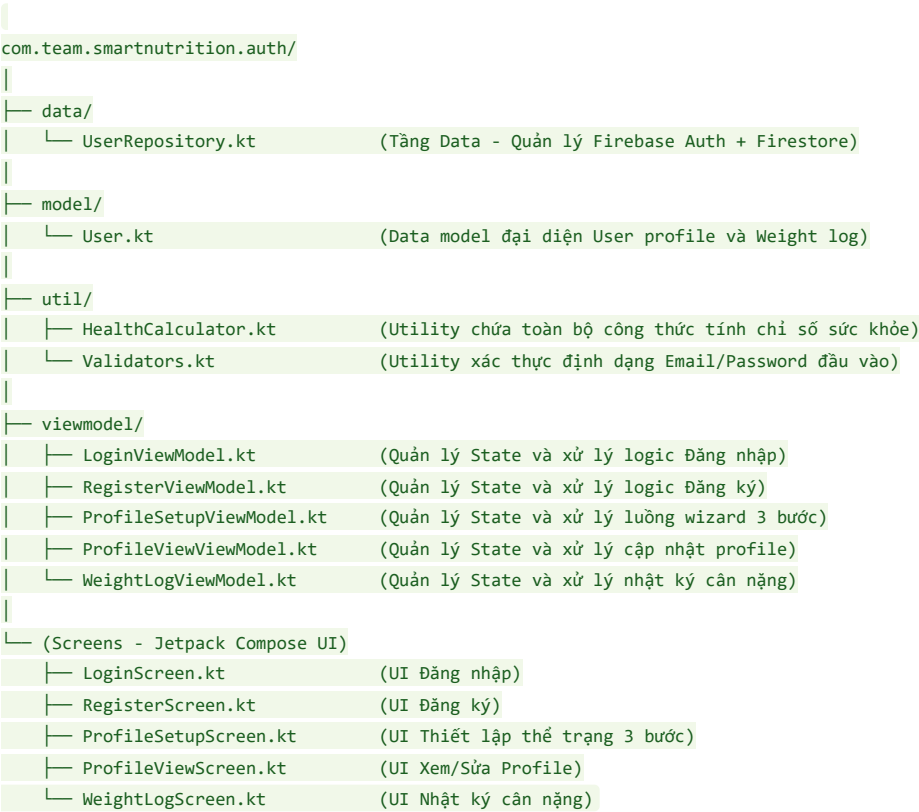
5. **WeightLogScreen:** Cho phép người dùng nhập cân nặng hôm nay, tự động cập nhật lại BMI và xem danh sách lịch sử cân nặng (sắp xếp giảm dần theo ngày).

2.2 Luồng hoạt động (User Flow)



3. DANH SÁCH CHI TIẾT CÁC FILE CODE & CLASS TRONG MODULE 1

Toàn bộ các file nằm trong package `com.team.smartnutrition.auth`.



3.1 Nhóm 1: Tầng dữ liệu (Model & Data Repository)

3.1.1 Class `User` & `WeightEntry` (trong file `User.kt`)

- **Mục đích:** Ảnh xạ dữ liệu trực tiếp 1:1 với tài liệu Firestore.
- **Class `User` :**
 - `uid: String` : Mã định danh người dùng từ Firebase Auth.
 - `email: String` : Địa chỉ email.
 - `displayName: String` : Tên hiển thị người dùng nhập vào.
 - `gender: String` : Giới tính (`"male"` | `"female"`).
 - `birthYear: Int` : Năm sinh (dùng để tính tuổi).
 - `heightCm: Int` : Chiều cao (cm).
 - `weightKg: Double` : Cân nặng hiện tại (kg).
 - `activityLevel: Double` : Hệ số vận động ($\$1.2 \rightarrow 1.9$).
 - `goal: String` : Mục tiêu sức khỏe (`"lose_weight"` | `"maintain"` | `"gain_muscle"`).
 - `bmi: Double` , `bmr: Double` , `tdee: Double` : Các chỉ số thể chất.
 - `proteinTarget: Int` , `carbTarget: Int` , `fatTarget: Int` : Lượng Macros mục tiêu (gram/ngày).
 - `calorieTarget: Int` : Lượng calo cần nạp hàng ngày (kcal/ngày).
 - `createdAt: Timestamp?` , `updatedAt: Timestamp?` : Thời gian tạo và cập nhật.
- **Class `WeightEntry` :** Đại diện cho 1 lần ghi nhận cân nặng trong sub-collection `weightLog` .
 - `weightKg: Double` : Cân nặng được ghi nhận.
 - `bmi: Double` : Chỉ số BMI tính lại dựa trên cân nặng mới này.
 - `loggedAt: Timestamp?` : Ngày giờ ghi nhận.
 - `date: String` : Định dạng `yyyy-MM-dd` (Đồng thời là Document ID).

3.1.2 Class `UserRepository` (trong `UserRepository.kt`)

- **Mục đích:** Đóng gói các phương thức của Firebase SDK.
- **Các thuộc tính:**
 - `auth = FirebaseAuth.getInstance()`
 - `firestore = FirebaseFirestore.getInstance()`
- **Các phương thức chính:**
 - `signInWithEmail(email, password)` : Xác thực email/password.
 - `signInWithCredential(credential)` : Đăng nhập Google (dùng Credential Manager).
 - `registerWithEmail(email, password)` : Tạo tài khoản mới.
 - `signOut()` : Đăng xuất người dùng.
 - `hasProfile(uid)` : Kiểm tra xem user có document trong collection `users` chưa.
 - `getUser(uid)` : Lấy dữ liệu user hiện tại.
 - `saveUser(user)` : Lưu/Cập nhật profile user lên Firestore.
 - `logWeight(uid, date, weight, bmi)` : Ghi nhận cân nặng mới (sử dụng **Batch Write** để ghi đè vào sub-collection `weightLog` đồng thời update trường `weightKg` và `bmi` ở user document gốc).
 - `getWeightHistory(uid, limit)` : Lấy danh sách lịch sử cân nặng từ `weightLog` , sắp xếp giảm dần theo thời gian.

3.2 Nhóm 2: Lớp nghiệp vụ & Tính toán (Utility Classes)

3.2.1 Object `HealthCalculator` (trong `HealthCalculator.kt`)

- **Mục đích:** Tính toán toàn bộ các chỉ số sức khỏe bằng các hàm thuần khiết (pure functions) chạy trực tiếp tại client.
- **Các phương thức chính:**
 - `calculateBmi(weightKg, heightCm)` : Trả về chỉ số BMI (\$Cân nặng / Chiều cao(m)^2\$).
 - `getBmiCategory(bmi)` : Phân loại thể trạng thành "Thiếu cân", "Bình thường", "Thừa cân", "Béo phì".
 - `calculateBmr(weightKg, heightCm, age, gender)` : Tính BMR theo công thức Harris-Benedict (phân chia nam/nữ).
 - `calculateTdee(bmr, activityLevel)` : Tính TDEE (\$BMR \times activityLevel\$).
 - `calculateCalorieTarget(tdee, goal)` : Điều chỉnh calo đích theo mục tiêu (Giảm mỡ: \$-500\$ kcal, Tăng cơ: \$+300\$ kcal, Duy trì: giữ nguyên).
 - `calculateMacros(calorieTarget, goal)` : Trả về bộ ba giá trị dinh dưỡng đạm/tinh bột/béo (Protein/Carb/Fat) theo tỷ lệ phần trăm năng lượng phù hợp.
 - `calculateAllMetrics(...)` : Hàm tổng hợp chạy 1 lượt tính hết các chỉ số trên \$\rightarrow\$ Trả về object `HealthMetrics`.

3.2.2 Object `Validators` (trong `Validators.kt`)

- **Mục đích:** Kiểm tra tính hợp lệ của dữ liệu người dùng nhập vào Form trước khi gửi lên Firebase.
- **Các phương thức chính:**
 - `isValidEmail(email)` : Sử dụng regex mặc định của Android (`Patterns.EMAIL_ADDRESS`).
 - `isValidPassword(password)` : Kiểm tra mật khẩu có \$ge 6\$ ký tự, có ít nhất 1 chữ viết hoa, và ít nhất 1 chữ số.
 - `doPasswordsMatch(password, confirmPassword)` : Đối chiếu tính trùng khớp.
 - `getEmailError()`, `getPasswordError()`, `getConfirmPasswordError()` : Trả về chuỗi thông báo lỗi cụ thể hiển thị trực tiếp lên TextField.

3.3 Nhóm 3: Tầng logic giao diện (ViewModels)

Các ViewModel sử dụng `StateFlow` để phát trạng thái (UI State) đến các Composable UI và sử dụng `viewModelScope` để chạy các tác vụ bất đồng bộ thông qua Kotlin Coroutines.

1. LoginViewModel:

- `uiState: StateFlow<LoginUiState>` : Chứa thông tin email, password nhập vào, trạng thái loading, lỗi, định hướng màn hình (`LoginDestination`).
- `signInWithEmail()` : Đăng nhập bằng Email/Password.
- `signInWithGoogle(activity)` : Kích hoạt Credential Manager lấy Google ID Token và chuyển đổi sang Firebase Credential để đăng nhập Google.
- `checkProfileAndNavigate(uid)` : Kiểm tra Firestore profile \$\rightarrow\$ Chọn điều hướng sang Home hay Setup.

2. RegisterViewModel:

- `uiState: StateFlow<RegisterUiState>` : Quản lý form đăng ký và lỗi hiển thị trực quan cho từng trường đầu vào.
- `register()` : Gọi repository để tạo tài khoản, nếu thành công đặt trạng thái `isRegistered = true`.

3. ProfileSetupViewModel:

- `uiState: StateFlow<ProfileSetupUiState>` : Quản lý biến bước hiện tại (`currentStep`), các dữ liệu nhập ở 3 bước (tên, giới tính, chiều cao, cân nặng, mục tiêu, vận động) và kết quả tính toán `HealthMetrics`.
- `nextStep()`, `previousStep()` : Chuyển bước trong giao diện Wizard. Khi chuyển sang bước 3, tự động gọi `recalculateMetrics()` để tính toán.
- `saveProfile()` : Tạo đối tượng `User` hoàn chỉnh và lưu lên Firestore thông qua repository.

4. ProfileViewViewModel:

- Đọc và hiển thị hồ sơ cá nhân hiện tại.
- Cập nhật và tính toán lại toàn bộ chỉ số khi người dùng sửa hồ sơ.

5. WeightLogViewModel:

- `uiState: StateFlow<WeightLogUiState>` : Chứa cân nặng nhập mới nhất, BMI tính realtime, danh sách lịch sử `history` .
- `saveWeight()` : Thực hiện lưu cân nặng hôm nay qua `repository.logWeight` .
- `loadWeightHistory()` : Nạp lịch sử cân nặng từ database.

4. THUẬT TOÁN VÀ CÔNG THỨC TÍNH TOÁN LOCAL

4.1 Chỉ số khối cơ thể (BMI)

$$BMI = \frac{WeightKg}{(HeightCm / 100)^2}$$

4.2 Tỷ lệ trao đổi chất cơ bản (BMR) - Công thức Harris-Benedict cải tiến

- **Nam giới:**

$$BMR = 88.362 + (13.397 \times WeightKg) + (4.799 \times HeightCm) - (5.677 \times Age)$$

- **Nữ giới:**

$$BMR = 447.593 + (9.247 \times WeightKg) + (3.098 \times HeightCm) - (4.330 \times Age)$$

4.3 Tổng lượng tiêu hao năng lượng hàng ngày (TDEE)

$$TDEE = BMR \times ActivityLevel$$

(Activity level tương ứng từ 1.2 đến 1.9)

4.4 Calorie Target (Năng lượng mục tiêu mỗi ngày)

- **Giảm mỡ (`lose_weight`)**: $TDEE - 500\text{ kcal}$
- **Tăng cơ (`gain_muscle`)**: $TDEE + 300\text{ kcal}$
- **Duy trì cân nặng (`maintain`)**: Giữ nguyên $TDEE$.

4.5 Macros Target (Tỷ lệ đạm, tinh bột, chất béo)

- **Tăng cơ**: 30% Protein (4 kcal/g), 45% Carb (4 kcal/g), 25% Fat (9 kcal/g).
- **Giảm mỡ**: 35% Protein, 35% Carb, 30% Fat.
- **Duy trì**: 25% Protein, 50% Carb, 25% Fat.

5. CẤU TRÚC FIRESTORE DATABASE (SCHEMA)

5.1 Document User: `users/{uid}`

```
{
  "uid": "1hF3Hj...",
  "email": "user@gmail.com",
  "displayName": "Nguyễn Văn A",
  "gender": "male",
  "birthYear": 2003,
  "heightCm": 170,
  "weightKg": 65.5,
  "activityLevel": 1.55,
  "goal": "lose_weight",
  "bmi": 22.66,
  "bmr": 1632.5,
  "tdee": 2530.4,
  "proteinTarget": 130,
  "carbTarget": 280,
  "fatTarget": 70,
  "calorieTarget": 2024,
  "createdAt": "2026-06-30T17:00:00Z",
  "updatedAt": "2026-07-01T01:00:00Z"
}
```

5.2 Collection Nhật ký Cân nặng: `users/{uid}/weightLog/{date}`

- **Document ID:** Định dạng `yyyy-MM-dd` (Ví dụ: `2026-07-01`).

```
{
  "weightKg": 65.2,
  "bmi": 22.56,
  "loggedAt": "2026-07-01T08:30:00Z"
}
```

6. HƯỚNG DẪN TRẢ LỜI CÁC CÂU HỎI CHỈNH SỬA CODE & LƯỒNG HOẠT ĐỘNG (Q&A CHO HỘI ĐỒNG)

Đây là các câu hỏi phổ biến nhất mà các thầy cô thường hỏi liên quan đến code và lập trình.

❓ Câu 1: "Thầy muốn thêm một trường mới vào hồ sơ sức khỏe, ví dụ dị ứng thực phẩm (`allergies`) hoặc bệnh nền (`medicalConditions`), em cần sửa những file nào và sửa ra sao?"

- **Trả lời:** Để thêm trường mới, chúng ta cần sửa qua 4 tầng code:

1. **Tầng Model (`User.kt`):** Khai báo thêm biến trong data class `User` (ví dụ: `val medicalConditions: String = ""`).

2. **Tầng UI State & ViewModel (`ProfileSetupViewModel.kt`):**

- Thêm biến `medicalConditions: String` vào data class `ProfileSetupUiState`.
- Tạo hàm cập nhật `fun updateMedicalConditions(text: String)` trong ViewModel để cập nhật State.
- Trong hàm `saveProfile()`, bổ sung trường `medicalConditions = state.medicalConditions` vào hàm dựng đối tượng `User` trước khi truyền sang Repository.

3. **Tầng Data (UserRepository.kt)**: Trong hàm `saveUser(user: User)`, bổ sung cặp khóa-giá trị `"medicalConditions"` to `user.medicalConditions` vào đối tượng HashMap để đẩy lên Firestore.

4. **Tầng UI (ProfileSetupScreen.kt)**: Tạo thêm một Composable TextField ở bước 3 (hoặc tạo thêm bước mới) để người dùng gõ/chọn thông tin bệnh nền, liên kết sự kiện `onValueChange` với hàm `viewModel.updateMedicalConditions(it)`.

? Câu 2: "Hiện tại quy tắc mật khẩu tối thiểu là 6 ký tự, có 1 chữ hoa, 1 chữ số. Nếu thầy muốn đổi thành: tối thiểu 8 ký tự, bắt buộc có ít nhất 1 ký tự đặc biệt (như @, #, \$,...) thì em sửa ở đâu?"

- **Trả lời**: Logic xác thực định dạng đầu vào được gom gọn hoàn toàn trong lớp tiện ích `Validators.kt`. Chúng ta chỉ cần chỉnh sửa tại đây mà không cần đụng đến UI hay ViewModel.

1. Mở file `Validators.kt`.

2. Tìm hàm `isValidPassword(password: String)` và cập nhật điều kiện:

```
fun isValidPassword(password: String): Boolean {
    val specialChars = setOf('@', '#', '$', '%', '^', '&', '*', '!', '_', '-')
    return password.length >= 8 && // Đổi từ 6 lên 8
        password.any { it.isUpperCase() } &&
        password.any { it.isDigit() } &&
        password.any { it in specialChars } // Thêm điều kiện ký tự đặc biệt
}
```

3. Tìm hàm `getPasswordError(password: String)` để cập nhật các câu thông báo lỗi tương ứng trả về cho giao diện (ví dụ đổi thành: "Mật khẩu phải có ít nhất 8 ký tự" và thêm thông báo "Cần ít nhất 1 ký tự đặc biệt").

? Câu 3: "Khi người dùng ấn nút đăng nhập bằng Google trên màn hình UI, luồng code chạy qua những đâu để đăng nhập thành công?"

- **Trả lời**: Luồng hoạt động chi tiết gồm các bước:

1. **UI (LoginScreen.kt)**: Người dùng click nút Google $\rightarrow$ UI gọi `context.findActivity()` để lấy context hiện tại và gọi `viewModel.signInWithGoogle(activity)`.

2. **ViewModel (LoginViewModel.kt)**: Chạy coroutine `viewModelScope.launch` gọi `CredentialManager` của hệ thống để hiển thị BottomSheet tài khoản Google của Android. Khi người dùng chọn tài khoản $\rightarrow$ Credential Manager trả về `GoogleIdTokenCredential` chứa chuỗi `idToken`.

3. **Chuyển đổi Credential**: ViewModel dùng `GoogleAuthProvider.getCredential(idToken, null)` để chuyển đổi ID Token của Google sang đối tượng `AuthCredential` phù hợp với Firebase.

4. **Repository (UserRepository.kt)**: ViewModel gọi `repository.signInWithCredential(credential)` $\rightarrow$ Repository thực hiện xác thực với Firebase SDK qua `.signInWithCredential(credential).await()` và trả về đối tượng `FirebaseUser`.

5. **Điều hướng**: Trở lại ViewModel, hàm `checkProfileAndNavigate(uid)` được kích hoạt. Nó kiểm tra xem UID này đã tồn tại trong collection `users` trên Firestore chưa. Nếu có $\rightarrow$ đặt trạng thái hướng đến là `LoginDestination.HOME`. Nếu chưa $\rightarrow$ chuyển hướng đến `LoginDestination.PROFILE_SETUP`.

? Câu 4: "Tại sao trong hàm lưu cân nặng (logWeight) ở UserRepository, em lại sử dụng Batch Write mà không ghi lần lượt? Code cụ thể chạy ra sao?"

- **Trả lời**:

1. *Lý do:* Khi người dùng cập nhật cân nặng mới, chúng ta cần ghi nhận nó vào **hai nơi khác nhau** trên Firestore: một bản ghi lịch sử ở sub-collection `users/{uid}/weightLog/{date}` và cập nhật trực tiếp hai trường `weightKg` và `bmi` tại document của user đó (`users/{uid}`) để các chức năng khác đọc được ngay lập tức. Nếu ghi lần lượt bằng 2 lệnh riêng biệt, khi mạng chập chờn hoặc có lỗi xảy ra, có thể lệnh 1 thành công nhưng lệnh 2 thất bại $\rightarrow$ dẫn đến dữ liệu không nhất quán.

2. *Cơ chế Batch Write:* Firebase Firestore cung cấp Batch Write để gộp các thao tác ghi lại thành một khối giao dịch nguyên tử (Atomic).

3. *Giải thích Code:*

```

val batch = firestore.batch()
// 1. Tạo tham chiếu và nạp lệnh ghi cân nặng ngày hôm nay
val weightRef = firestore.collection("users").document(uid)
    .collection("weightLog").document(date)
batch.set(weightRef, entry)

// 2. Tạo tham chiếu và nạp lệnh cập nhật cân nặng & bmi tại user profile
val userRef = firestore.collection("users").document(uid)
batch.update(userRef, mapOf(
    "weightKg" to weightKg,
    "bmi" to bmi,
    "updatedAt" to Timestamp.now()
))

// 3. Thực hiện commit toàn bộ batch. Cả hai ghi thành công hoặc cùng thất bại
batch.commit()

```

? Câu 5: "Lớp `HealthCalculator` là class thông thường hay là gì? Tại sao em khai báo là `object`?"

- **Trả lời:** Trong Kotlin, `object` định nghĩa một lớp Singleton (chỉ có duy nhất một thực thể trong suốt vòng đời ứng dụng). Lớp `HealthCalculator` chỉ chứa các hàm tiện ích tính toán (hàm static helper), không lưu trữ trạng thái (state-less) của một đối tượng cụ thể nào. Việc dùng `object` giúp chúng ta gọi trực tiếp các phương thức như `HealthCalculator.calculateBmi(...)` từ bất kỳ đâu mà không cần tốn bộ nhớ khởi tạo đối tượng mới (không cần dùng từ khóa `new` như Java).

MÃ NGUỒN CHI TIẾT MODULE 1 (SOURCE CODE)

Dưới đây là nội dung toàn bộ mã nguồn của các lớp thuộc Module 1, sắp xếp theo cấu trúc thư mục dự án.

File: auth/LoginScreen.kt

```

package com.team.smartnutrition.auth

import android.app.Activity
import androidx.compose.animation.AnimatedVisibility
import androidx.compose.animation.fadeIn
import androidx.compose.animation.fadeOut
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.text.KeyboardActions
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.foundation.verticalScroll
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Email
import androidx.compose.material.icons.filled.Lock
import androidx.compose.material.icons.filled.Visibility
import androidx.compose.material.icons.filled.VisibilityOff
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.focus.FocusDirection
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.platform.LocalFocusManager
import androidx.compose.ui.res.stringResource
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.input.ImeAction
import androidx.compose.ui.text.input.KeyboardType
import androidx.compose.ui.text.input.PasswordVisualTransformation
import androidx.compose.ui.text.input.VisualTransformation
import androidx.compose.ui.text.style.TextAlign
import androidx.compose.ui.unit.dp
import androidx.lifecycle.viewmodel.compose.viewModel
import androidx.navigation.NavController
import android.content.Context
import android.content.ContextWrapper
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.res.painterResource
import com.team.smartnutrition.R
import com.team.smartnutrition.auth.viewmodel.LoginDestination
import com.team.smartnutrition.auth.viewmodel.LoginViewModel
import com.team.smartnutrition.navigation.Screen

private fun Context.findActivity(): Activity? {
    var currentContext = this
    while (currentContext is ContextWrapper) {
        if (currentContext is Activity) {
            return currentContext
        }
        currentContext = currentContext.baseContext
    }
    return null
}

/**
 * =====
 * MODULE 1 - TV1: MÀN HÌNH ĐĂNG NHẬP
 * =====
 *
 * Tính năng:
 * - Đăng nhập Email/Password (Firebase Auth)
 * - Đăng nhập Google (Credential Manager)

```

```

* - Kiểm tra profile → navigate phù hợp
* - Validation + error handling tiếng Việt/Anh
*/
@Composable
fun LoginScreen(
    navController: NavController,
    viewModel: LoginViewModel = viewModel()
) {
    val uiState by viewModel.uiState.collectAsState()
    var passwordVisible by remember { mutableStateOf(false) }
    val focusManager = LocalFocusManager.current
    val context = LocalContext.current
    val snackbarHostState = remember { SnackbarHostState() }

    // Handle navigation
    LaunchedEffect(uiState.navigateTo) {
        uiState.navigateTo?.let { destination ->
            val route = when (destination) {
                LoginDestination.HOME -> Screen.Home.route
                LoginDestination.PROFILE_SETUP -> Screen.ProfileSetup.route
            }
            navController.navigate(route) {
                popUpTo(Screen.Login.route) { inclusive = true }
            }
            viewModel.onNavigated()
        }
    }

    // Handle error snackbar
    LaunchedEffect(uiState.errorMessage) {
        uiState.errorMessage?.let { message ->
            snackbarHostState.showSnackbar(message)
            viewModel.clearError()
        }
    }

    Scaffold(
        snackbarHost = { SnackbarHost(snackbarHostState) }
    ) { padding ->
        Column(
            modifier = Modifier
                .fillMaxSize()
                .padding(padding)
                .padding(24.dp)
                .verticalScroll(rememberScrollState()),
            horizontalAlignment = Alignment.CenterHorizontally,
            verticalArrangement = Arrangement.Center
        ) {
            Spacer(modifier = Modifier.weight(1f))

            // == Logo & App Name ==
            Text(
                text = "🍎",
                style = MaterialTheme.typography.displayMedium,
                modifier = Modifier.padding(bottom = 8.dp)
            )
            Text(
                text = "Smart Nutrition",
                style = MaterialTheme.typography.headlineLarge.copy(
                    fontWeight = FontWeight.Bold
                ),
                color = MaterialTheme.colorScheme.primary
            )
        }
    }
}

```

```

    )
    Text(
        text = stringResource(R.string.login_subtitle),
        style = MaterialTheme.typography.bodyMedium,
        color = MaterialTheme.colorScheme.onSurfaceVariant,
        textAlign = TextAlign.Center,
        modifier = Modifier.padding(top = 4.dp, bottom = 40.dp)
    )

```

```

// == Email Field ==
OutlinedTextField(
    value = uiState.email,
    onChange = { viewModel.updateEmail(it) },
    label = { Text(stringResource(R.string.email_label)) },
    leadingIcon = {
        Icon(Icons.Filled.Email, contentDescription = null)
    },
    keyboardOptions = KeyboardOptions(
        keyboardType = KeyboardType.Email,
        imeAction = ImeAction.Next
    ),
    keyboardActions = KeyboardActions(
        onNext = { focusManager.moveFocus(FocusDirection.Down) }
    ),
    shape = RoundedCornerShape(12.dp),
    modifier = Modifier.fillMaxWidth(),
    singleLine = true,
    enabled = !uiState.isLoading
)
Spacer(modifier = Modifier.height(12.dp))

```

```

// == Password Field ==
OutlinedTextField(
    value = uiState.password,
    onChange = { viewModel.updatePassword(it) },
    label = { Text(stringResource(R.string.password_label)) },
    leadingIcon = {
        Icon(Icons.Filled.Lock, contentDescription = null)
    },
    trailingIcon = {
        IconButton(onClick = { passwordVisible = !passwordVisible }) {
            Icon(
                if (passwordVisible) Icons.Filled.Visibility
                else Icons.Filled.VisibilityOff,
                contentDescription = stringResource(R.string.password_label)
            )
        }
    },
    visualTransformation = if (passwordVisible) VisualTransformation.None
    else PasswordVisualTransformation(),
    keyboardOptions = KeyboardOptions(
        keyboardType = KeyboardType.Password,
        imeAction = ImeAction.Done
    ),
    keyboardActions = KeyboardActions(
        onDone = {
            focusManager.clearFocus()
            if (viewModel.isFormValid()) viewModel.signInWithEmail()
        }
    ),
    shape = RoundedCornerShape(12.dp),
    modifier = Modifier.fillMaxWidth(),

```

```

        singleLine = true,
        enabled = !uiState.isLoading
    )
    Spacer(modifier = Modifier.height(24.dp))

    // == Login Button ==
    Button(
        onClick = { viewModel.signInWithEmail() },
        modifier = Modifier
            .fillMaxWidth()
            .height(52.dp),
        shape = RoundedCornerShape(12.dp),
        enabled = !uiState.isLoading && viewModel.isFormValid()
    ) {
        if (uiState.isLoading) {
            CircularProgressIndicator(
                modifier = Modifier.size(20.dp),
                color = MaterialTheme.colorScheme.onPrimary,
                strokeWidth = 2.dp
            )
        } else {
            Text(stringResource(R.string.login_button), style = MaterialTheme.typography.labelLarge)
        }
    }
    Spacer(modifier = Modifier.height(12.dp))

    // == Divider ==
    Row(
        modifier = Modifier.fillMaxWidth(),
        verticalAlignment = Alignment.CenterVertically
    ) {
        HorizontalDivider(modifier = Modifier.weight(1f))
        Text(
            text = " " + stringResource(R.string.or_divider) + " ",
            style = MaterialTheme.typography.bodySmall,
            color = MaterialTheme.colorScheme.onSurfaceVariant
        )
        HorizontalDivider(modifier = Modifier.weight(1f))
    }
    Spacer(modifier = Modifier.height(12.dp))

    // == Google Sign-In Button ==
    OutlinedButton(
        onClick = {
            val activity = context.findActivity()
            if (activity != null) {
                viewModel.signInWithGoogle(activity)
            }
        },
        modifier = Modifier
            .fillMaxWidth()
            .height(52.dp),
        shape = RoundedCornerShape(12.dp),
        enabled = !uiState.isLoading
    ) {
        Icon(
            painter = painterResource(id = R.drawable.ic_google),
            contentDescription = "Google Logo",
            modifier = Modifier.size(20.dp),
            tint = Color.Unspecified
        )
        Spacer(modifier = Modifier.width(12.dp))
    }

```

```
        Text(  
            text = stringResource(R.string.google_signin),  
            style = MaterialTheme.typography.labelLarge  
        )  
    }  
  
    Spacer(modifier = Modifier.weight(1f))  
  
    // == Register Link ==  
    TextButton(  
        onClick = { navController.navigate(Screen.Register.route) },  
        enabled = !uiState.isLoading  
    ) {  
        Text(  
            stringResource(R.string.no_account),  
            color = MaterialTheme.colorScheme.primary  
        )  
    }  
    Spacer(modifier = Modifier.height(16.dp))  
}  
}
```

File: auth/ProfileSetupScreen.kt

```

package com.team.smartnutrition.auth

import androidx.compose.animation.*
import androidx.compose.foundation.BorderStroke
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.verticalScroll
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.automirrored.filled.ArrowBack
import androidx.compose.material.icons.automirrored.filled.ArrowForward
import androidx.compose.material.icons.filled.*
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.res.stringResource
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.style.TextAlign
import androidx.compose.ui.unit.dp
import androidx.lifecycle.viewmodel.compose.viewModel
import androidx.navigation.NavController
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.ui.text.input.KeyboardType
import androidx.compose.ui.text.input.ImeAction
import com.team.smartnutrition.R
import com.team.smartnutrition.auth.viewmodel.*
import com.team.smartnutrition.navigation.Screen
import java.time.Year

/**
 * =====
 * MODULE 1 - TV1: THIẾT LẬP THỂ TRẠNG
 * =====
 *
 * Multi-step wizard 3 bước:
 * Step 1: Giới tính + Năm sinh + Tên hiển thị
 * Step 2: Chiều cao + Cân nặng (Slider + Input)
 * Step 3: Mục tiêu + Mức vận động + Kết quả tính toán
 */
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun ProfileSetupScreen(
    navController: NavController,
    viewModel: ProfileSetupViewModel = viewModel()
) {
    val uiState by viewModel.uiState.collectAsState()
    val snackbarHostState = remember { SnackbarHostState() }

    // Handle navigation after save
    LaunchedEffect(uiState.isSaved) {
        if (uiState.isSaved) {
            navController.navigate(Screen.Home.route) {
                popUpTo(Screen.Login.route) { inclusive = true }
            }
            viewModel.onNavigated()
        }
    }

    // Handle error
    LaunchedEffect(uiState.errorMessage) {
        uiState.errorMessage?.let { snackbarHostState.showSnackbar(it) }
    }
}

```



```

    }

    Scaffold(
        topBar = {
            TopAppBar(
                title = {
                    Text(stringResource(R.string.profile_setup_step_title, uiState.currentStep, uiState.totalSteps))
                },
                colors = TopAppBarDefaults.topAppBarColors(
                    containerColor = MaterialTheme.colorScheme.surface
                )
            )
        },
        snackbarHost = { SnackbarHost(snackbarHostState) }
    ) { padding ->
        Column(
            modifier = Modifier
                .fillMaxSize()
                .padding(padding)
        ) {
            // == Progress Bar ==
            LinearProgressIndicator(
                progress = { uiState.currentStep.toFloat() / uiState.totalSteps },
                modifier = Modifier
                    .fillMaxWidth()
                    .height(4.dp),
                color = MaterialTheme.colorScheme.primary,
                trackColor = MaterialTheme.colorScheme.surfaceVariant,
            )

            // == Step Content ==
            Column(
                modifier = Modifier
                    .weight(1f)
                    .verticalScroll(rememberScrollState())
                    .padding(24.dp)
            ) {
                AnimatedContent(
                    targetState = uiState.currentStep,
                    transitionSpec = {
                        if (targetState > initialState) {
                            slideInHorizontally { width -> width } + fadeIn() togetherWith
                                slideOutHorizontally { width -> -width } + fadeOut()
                        } else {
                            slideInHorizontally { width -> -width } + fadeIn() togetherWith
                                slideOutHorizontally { width -> width } + fadeOut()
                        }
                    },
                    label = "stepTransition"
                ) { step ->
                    when (step) {
                        1 -> Step1Content(uiState, viewModel)
                        2 -> Step2Content(uiState, viewModel)
                        3 -> Step3Content(uiState, viewModel)
                    }
                }
            }
        }

        // == Bottom Buttons ==
        Surface(
            tonalElevation = 2.dp,
            modifier = Modifier.fillMaxWidth()
        )
    }

```

```

    ) {
        Row(
            modifier = Modifier
                .fillMaxWidth()
                .padding(16.dp),
            horizontalArrangement = Arrangement.spacedBy(12.dp)
        ) {
            // Back button
            if (uiState.currentStep > 1) {
                OutlinedButton(
                    onClick = { viewModel.previousStep() },
                    modifier = Modifier
                        .weight(1f)
                        .height(52.dp),
                    shape = RoundedCornerShape(12.dp)
                ) {
                    Icon(
                        Icons.AutoMirrored.Filled.ArrowBack,
                        contentDescription = null,
                        modifier = Modifier.size(18.dp)
                    )
                    Spacer(modifier = Modifier.width(8.dp))
                    Text(stringResource(R.string.back))
                }
            }

            // Next / Save button
            Button(
                onClick = {
                    if (uiState.currentStep < uiState.totalSteps) {
                        viewModel.nextStep()
                    } else {
                        viewModel.saveProfile()
                    }
                },
                modifier = Modifier
                    .weight(1f)
                    .height(52.dp),
                shape = RoundedCornerShape(12.dp),
                enabled = viewModel.isCurrentStepValid() && !uiState.isSaving
            ) {
                if (uiState.isSaving) {
                    CircularProgressIndicator(
                        modifier = Modifier.size(20.dp),
                        color = MaterialTheme.colorScheme.onPrimary,
                        strokeWidth = 2.dp
                    )
                } else if (uiState.currentStep < uiState.totalSteps) {
                    Text(stringResource(R.string.next))
                    Spacer(modifier = Modifier.width(8.dp))
                    Icon(
                        Icons.AutoMirrored.Filled.ArrowForward,
                        contentDescription = null,
                        modifier = Modifier.size(18.dp)
                    )
                } else {
                    Icon(
                        Icons.Filled.Check,
                        contentDescription = null,
                        modifier = Modifier.size(18.dp)
                    )
                    Spacer(modifier = Modifier.width(8.dp))
                }
            }
        }
    }

```

```

        Text(stringResource(R.string.finish))
    }
}

// =====
// STEP 1: Giới tính + Năm sinh + Tên
// =====
@Composable
private fun Step1Content(
    uiState: ProfileSetupUiState,
    viewModel: ProfileSetupViewModel
) {
    Column(verticalArrangement = Arrangement.spacedBy(24.dp)) {
        // Title
        Text(
            text = stringResource(R.string.basic_info_title),
            style = MaterialTheme.typography.headlineSmall.copy(fontWeight = FontWeight.Bold)
        )
        Text(
            text = stringResource(R.string.basic_info_subtitle),
            style = MaterialTheme.typography.bodyMedium,
            color = MaterialTheme.colorScheme.onSurfaceVariant
        )

        // == Tên hiển thị ==
        OutlinedTextField(
            value = uiState.displayName,
            onChange = { viewModel.updateDisplayName(it) },
            label = { Text(stringResource(R.string.display_name_label)) },
            leadingIcon = { Icon(Icons.Filled.Person, contentDescription = null) },
            shape = RoundedCornerShape(12.dp),
            modifier = Modifier.fillMaxWidth(),
            singleLine = true
        )

        // == Giới tính ==
        Text(
            text = stringResource(R.string.gender_label),
            style = MaterialTheme.typography.titleMedium
        )
        Row(
            modifier = Modifier.fillMaxWidth(),
            horizontalArrangement = Arrangement.spacedBy(12.dp)
        ) {
            GenderCard(
                emoji = "♂",
                label = stringResource(R.string.gender_male),
                isSelected = uiState.gender == "male",
                onClick = { viewModel.updateGender("male") },
                modifier = Modifier.weight(1f)
            )
            GenderCard(
                emoji = "♀",
                label = stringResource(R.string.gender_female),
                isSelected = uiState.gender == "female",
                onClick = { viewModel.updateGender("female") },
                modifier = Modifier.weight(1f)
            )
        }
    }
}

```

```

    )
}

// == Năm sinh ==
Text(
    text = stringResource(R.string.birth_year_display, uiState.birthYear, Year.now().value - uiState.birthYear),
    style = MaterialTheme.typography.titleMedium
)
Slider(
    value = uiState.birthYear.toFloat(),
    onValueChange = { viewModel.updateBirthYear(it.toInt()) },
    valueRange = 1940f..2015f,
    steps = 74,
    modifier = Modifier.fillMaxWidth()
)
Row(
    modifier = Modifier.fillMaxWidth(),
    horizontalArrangement = Arrangement.SpaceBetween
) {
    Text("1940", style = MaterialTheme.typography.bodySmall,
        color = MaterialTheme.colorScheme.onSurfaceVariant)
    Text("2015", style = MaterialTheme.typography.bodySmall,
        color = MaterialTheme.colorScheme.onSurfaceVariant)
}

var birthYearInputText by remember { mutableStateOf(uiState.birthYear.toString()) }
LaunchedEffect(uiState.birthYear) {
    if (birthYearInputText.toIntOrNull() != uiState.birthYear) {
        birthYearInputText = uiState.birthYear.toString()
    }
}

OutlinedTextField(
    value = birthYearInputText,
    onValueChange = { newValue ->
        if (newValue.isEmpty() || newValue.all { it.isDigit() }) {
            birthYearInputText = newValue
            newValue.toIntOrNull()?.let { year ->
                if (year in 1940..2015) {
                    viewModel.updateBirthYear(year)
                }
            }
        }
    },
    label = { Text(stringResource(R.string.birth_year_label, uiState.birthYear).substringBefore(":") + " (1940 - 2015)") },
    keyboardOptions = KeyboardOptions(
        keyboardType = KeyboardType.Number,
        imeAction = ImeAction.Next
    ),
    shape = RoundedCornerShape(12.dp),
    modifier = Modifier.fillMaxWidth(),
    singleLine = true
)
}
}

```

```

@Composable
private fun GenderCard(
    emoji: String,
    label: String,
    isSelected: Boolean,
    onClick: () -> Unit,

```

```

        modifier: Modifier = Modifier
    ) {
        Card(
            onClick = onClick,
            modifier = modifier.height(100.dp),
            shape = RoundedCornerShape(16.dp),
            colors = CardDefaults.cardColors(
                containerColor = if (isSelected) MaterialTheme.colorScheme.primaryContainer
                else MaterialTheme.colorScheme.surfaceVariant
            ),
            border = if (isSelected) BorderStroke(2.dp, MaterialTheme.colorScheme.primary)
            else null
        ) {
            Column(
                modifier = Modifier.fillMaxSize(),
                horizontalAlignment = Alignment.CenterHorizontally,
                verticalArrangement = Arrangement.Center
            ) {
                Text(text = emoji, style = MaterialTheme.typography.headlineMedium)
                Spacer(modifier = Modifier.height(4.dp))
                Text(
                    text = label,
                    style = MaterialTheme.typography.labelLarge,
                    color = if (isSelected) MaterialTheme.colorScheme.primary
                    else MaterialTheme.colorScheme.onSurfaceVariant
                )
            }
        }
    }
}

// =====
// STEP 2: Chiều cao + Cân nặng
// =====
@Composable
private fun Step2Content(
    uiState: ProfileSetupUiState,
    viewModel: ProfileSetupViewModel
) {
    var heightInputText by remember { mutableStateOf(uiState.heightCm.toString()) }
    LaunchedEffect(uiState.heightCm) {
        if (heightInputText.toIntOrNull() != uiState.heightCm) {
            heightInputText = uiState.heightCm.toString()
        }
    }

    var weightInputText by remember { mutableStateOf("%.1f".format(uiState.weightKg)) }
    LaunchedEffect(uiState.weightKg) {
        val currentFloat = weightInputText.toDoubleOrNull() ?: 0.0
        if (Math.abs(currentFloat - uiState.weightKg) > 0.05) {
            weightInputText = "%.1f".format(uiState.weightKg)
        }
    }

    Column(verticalArrangement = Arrangement.spacedBy(24.dp)) {
        Text(
            text = stringResource(R.string.body_metrics_title),
            style = MaterialTheme.typography.headlineSmall.copy(fontWeight = FontWeight.Bold)
        )
        Text(
            text = stringResource(R.string.body_metrics_subtitle),
            style = MaterialTheme.typography.bodyMedium,
            color = MaterialTheme.colorScheme.onSurfaceVariant
        )
    }
}

```

```

    )

    // == Chiều cao ==
    Card(
        modifier = Modifier.fillMaxWidth(),
        shape = RoundedCornerShape(16.dp),
        colors = CardDefaults.cardColors(
            containerColor = MaterialTheme.colorScheme.surfaceVariant
        )
    ) {
        Column(modifier = Modifier.padding(20.dp)) {
            Row(
                modifier = Modifier.fillMaxWidth(),
                horizontalArrangement = Arrangement.SpaceBetween,
                verticalAlignment = Alignment.CenterVertically
            ) {
                Text("📏 " + stringResource(R.string.height_label), style = MaterialTheme.typography.titleMedium)
                Text(
                    text = "${uiState.heightCm} cm",
                    style = MaterialTheme.typography.headlineSmall.copy(
                        fontWeight = FontWeight.Bold
                    ),
                    color = MaterialTheme.colorScheme.primary
                )
            }
            Spacer(modifier = Modifier.height(12.dp))
            Slider(
                value = uiState.heightCm.toFloat(),
                onValueChange = { viewModel.updateHeightCm(it.toInt()) },
                valueRange = 100f..250f,
                modifier = Modifier.fillMaxWidth()
            )
            Row(
                modifier = Modifier.fillMaxWidth(),
                horizontalArrangement = Arrangement.SpaceBetween
            ) {
                Text("100 cm", style = MaterialTheme.typography.bodySmall,
                    color = MaterialTheme.colorScheme.onSurfaceVariant)
                Text("250 cm", style = MaterialTheme.typography.bodySmall,
                    color = MaterialTheme.colorScheme.onSurfaceVariant)
            }
            Spacer(modifier = Modifier.height(12.dp))
            OutlinedTextField(
                value = heightInputText,
                onValueChange = { newValue ->
                    if (newValue.isEmpty() || newValue.all { it.isDigit() }) {
                        heightInputText = newValue
                        newValue.toIntOrNull()?.let { height ->
                            if (height in 100..250) {
                                viewModel.updateHeightCm(height)
                            }
                        }
                    }
                }
            ),
            label = { Text("Nhập chiều cao thủ công") },
            suffix = { Text("cm") },
            keyboardOptions = KeyboardOptions(
                keyboardType = KeyboardType.Number,
                imeAction = ImeAction.Next
            ),
            shape = RoundedCornerShape(12.dp),
            modifier = Modifier.fillMaxWidth(),

```

```

        singleLine = true,
        colors = OutlinedTextFieldDefaults.colors(
            unfocusedContainerColor = MaterialTheme.colorScheme.surface,
            focusedContainerColor = MaterialTheme.colorScheme.surface
        )
    )
}

// == Cân nặng ==
Card(
    modifier = Modifier.fillMaxWidth(),
    shape = RoundedCornerShape(16.dp),
    colors = CardDefaults.cardColors(
        containerColor = MaterialTheme.colorScheme.surfaceVariant
    )
) {
    Column(modifier = Modifier.padding(20.dp)) {
        Row(
            modifier = Modifier.fillMaxWidth(),
            horizontalArrangement = Arrangement.SpaceBetween,
            verticalAlignment = Alignment.CenterVertically
        ) {
            Text("🏋️ " + stringResource(R.string.weight_label), style = MaterialTheme.typography.titleMedium)
            Text(
                text = "${"%1f".format(uiState.weightKg)} kg",
                style = MaterialTheme.typography.headlineSmall.copy(
                    fontWeight = FontWeight.Bold
                ),
                color = MaterialTheme.colorScheme.primary
            )
        }
        Spacer(modifier = Modifier.height(12.dp))
        Slider(
            value = uiState.weightKg.toFloat(),
            onValueChange = {
                viewModel.updateWeightKg(
                    (it * 10).toInt() / 10.0 // Round to 0.1
                )
            },
            valueRange = 30f..200f,
            modifier = Modifier.fillMaxWidth()
        )
        Row(
            modifier = Modifier.fillMaxWidth(),
            horizontalArrangement = Arrangement.SpaceBetween
        ) {
            Text("30 kg", style = MaterialTheme.typography.bodySmall,
                color = MaterialTheme.colorScheme.onSurfaceVariant)
            Text("200 kg", style = MaterialTheme.typography.bodySmall,
                color = MaterialTheme.colorScheme.onSurfaceVariant)
        }
        Spacer(modifier = Modifier.height(12.dp))
        OutlinedTextField(
            value = weightInputText,
            onValueChange = { newValue ->
                val normalized = newValue.replace(',', '.')
                if (normalized.isEmpty() || normalized.toDoubleOrNull() != null || normalized == "." || normalized.endsWith(".")) {
                    weightInputText = newValue
                    normalized.toDoubleOrNull()?.let { weight ->
                        if (weight >= 30.0 && weight <= 200.0) {
                            viewModel.updateWeightKg(weight)
                        }
                    }
                }
            }
        )
    }
}

```

```

    }
    }
    },
    label = { Text("Nhập cân nặng thủ công") },
    suffix = { Text("kg") },
    keyboardOptions = KeyboardOptions(
        keyboardType = KeyboardType.Decimal,
        imeAction = ImeAction.Done
    ),
    shape = RoundedCornerShape(12.dp),
    modifier = Modifier.fillMaxWidth(),
    singleLine = true,
    colors = OutlinedTextFieldDefaults.colors(
        unfocusedContainerColor = MaterialTheme.colorScheme.surface,
        focusedContainerColor = MaterialTheme.colorScheme.surface
    )
    )
}
}

// == Preview BMI ==
val bmi = com.team.smartnutrition.auth.util.HealthCalculator.calculateBmi(
    uiState.weightKg, uiState.heightCm
)
val bmiCategory = com.team.smartnutrition.auth.util.HealthCalculator.getBmiCategory(bmi)

Card(
    modifier = Modifier.fillMaxWidth(),
    shape = RoundedCornerShape(16.dp),
    colors = CardDefaults.cardColors(
        containerColor = MaterialTheme.colorScheme.primaryContainer
    )
) {
    Row(
        modifier = Modifier
            .fillMaxWidth()
            .padding(16.dp),
        horizontalArrangement = Arrangement.SpaceBetween,
        verticalAlignment = Alignment.CenterVertically
    ) {
        Column {
            Text(
                stringResource(R.string.current_bmi),
                style = MaterialTheme.typography.bodyMedium,
                color = MaterialTheme.colorScheme.onPrimaryContainer
            )
            Text(
                bmiCategory,
                style = MaterialTheme.typography.bodySmall,
                color = MaterialTheme.colorScheme.onPrimaryContainer
            )
        }
        Text(
            "${"%1f".format(bmi)}",
            style = MaterialTheme.typography.headlineMedium.copy(
                fontWeight = FontWeight.Bold
            ),
            color = MaterialTheme.colorScheme.primary
        )
    }
}
}

```



```

    }
}

// =====
// STEP 3: Mục tiêu + Vận động + Kết quả
// =====
@Composable
private fun Step3Content(
    uiState: ProfileSetupUiState,
    viewModel: ProfileSetupViewModel
) {
    Column(verticalArrangement = Arrangement.spacedBy(20.dp)) {
        Text(
            text = stringResource(R.string.goal_activity_title),
            style = MaterialTheme.typography.headlineSmall.copy(fontWeight = FontWeight.Bold)
        )

        // == Mục tiêu sức khỏe ==
        Text(stringResource(R.string.goal_label), style = MaterialTheme.typography.titleMedium)
        goalOptions.forEach { option ->
            val labelText = when (option.value) {
                "lose_weight" -> stringResource(R.string.goal_lose)
                "gain_muscle" -> stringResource(R.string.goal_gain)
                else -> stringResource(R.string.goal_maintain)
            }
            val descText = when (option.value) {
                "lose_weight" -> stringResource(R.string.goal_lose_desc)
                "gain_muscle" -> stringResource(R.string.goal_gain_desc)
                else -> stringResource(R.string.goal_maintain_desc)
            }
            Card(
                onClick = { viewModel.updateGoal(option.value) },
                modifier = Modifier.fillMaxWidth(),
                shape = RoundedCornerShape(12.dp),
                colors = CardDefaults.cardColors(
                    containerColor = if (uiState.goal == option.value)
                        MaterialTheme.colorScheme.primaryContainer
                    else MaterialTheme.colorScheme.surfaceVariant
                ),
                border = if (uiState.goal == option.value)
                    BorderStroke(2.dp, MaterialTheme.colorScheme.primary)
                else null
            ) {
                Row(
                    modifier = Modifier
                        .fillMaxWidth()
                        .padding(16.dp),
                    verticalAlignment = Alignment.CenterVertically
                ) {
                    Text(option.emoji, style = MaterialTheme.typography.headlineSmall)
                    Spacer(modifier = Modifier.width(12.dp))
                    Column(modifier = Modifier.weight(1f)) {
                        Text(
                            labelText,
                            style = MaterialTheme.typography.titleMedium,
                            fontWeight = FontWeight.SemiBold
                        )
                        Text(
                            descText,
                            style = MaterialTheme.typography.bodySmall,
                            color = MaterialTheme.colorScheme.onSurfaceVariant
                        )
                    }
                }
            }
        }
    }
}

```

```

    }
    if (uiState.goal == option.value) {
        Icon(
            Icons.Filled.CheckCircle,
            contentDescription = null,
            tint = MaterialTheme.colorScheme.primary
        )
    }
}
}

Spacer(modifier = Modifier.height(4.dp))

// == Mức độ vận động ==
Text(stringResource(R.string.activity_label), style = MaterialTheme.typography.titleMedium)
activityLevelOptions.forEach { option ->
    val labelText = when (option.value) {
        1.2 -> stringResource(R.string.activity_sedentary)
        1.375 -> stringResource(R.string.activity_light)
        1.55 -> stringResource(R.string.activity_moderate)
        1.725 -> stringResource(R.string.activity_very)
        else -> stringResource(R.string.activity_extreme)
    }
    Row(
        modifier = Modifier.fillMaxWidth(),
        verticalAlignment = Alignment.CenterVertically
    ) {
        RadioButton(
            selected = uiState.activityLevel == option.value,
            onClick = { viewModel.updateActivityLevel(option.value) }
        )
        Text(
            text = "${option.emoji} $labelText",
            style = MaterialTheme.typography.bodyLarge,
            modifier = Modifier.padding(start = 4.dp)
        )
    }
}

// == Kết quả tính toán ==
uiState.metrics?.let { metrics ->
    HorizontalDivider(modifier = Modifier.padding(vertical = 8.dp))
    Text(
        text = stringResource(R.string.your_metrics_title),
        style = MaterialTheme.typography.titleMedium.copy(fontWeight = FontWeight.Bold)
    )

    Card(
        modifier = Modifier.fillMaxWidth(),
        shape = RoundedCornerShape(16.dp),
        colors = CardDefaults.cardColors(
            containerColor = MaterialTheme.colorScheme.primaryContainer
        )
    ) {
        Column(modifier = Modifier.padding(16.dp)) {
            // BMI & BMR
            Row(
                modifier = Modifier.fillMaxWidth(),
                horizontalArrangement = Arrangement.SpaceEvenly
            ) {
                MetricItem("BMI", "${"%1f".format(metrics.bmi)}", metrics.bmiCategory)
            }
        }
    }
}

```

```
@Composable
private fun MetricItem(label: String, value: String, unit: String) {
    Column(horizontalAlignment = Alignment.CenterHorizontally) {
        Text(
            text = label,
            style = MaterialTheme.typography.bodySmall,
            color = MaterialTheme.colorScheme.onPrimaryContainer.copy(alpha = 0.7f)
        )
        Text(
            text = value,
            style = MaterialTheme.typography.titleLarge.copy(fontWeight = FontWeight.Bold),
            color = MaterialTheme.colorScheme.primary
        )
        if (unit.isNotBlank()) {
            Text(
                text = unit,
                style = MaterialTheme.typography.bodySmall,
                color = MaterialTheme.colorScheme.onPrimaryContainer.copy(alpha = 0.5f)
            )
        }
    }
}
```

27/69

```
package com.team.smartnutrition.auth
```

```
import androidx.compose.foundation.layout.*
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.shape.CircleShape
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.verticalScroll
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.*
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.draw.clip
import androidx.compose.ui.res.stringResource
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.unit.dp
import androidx.lifecycle.viewmodel.compose.viewModel
import androidx.navigation.NavController
import com.team.smartnutrition.R
import com.team.smartnutrition.auth.util.HealthCalculator
import com.team.smartnutrition.auth.viewmodel.ProfileViewUiState
import com.team.smartnutrition.auth.viewmodel.ProfileViewViewModel
import com.team.smartnutrition.auth.viewmodel.activityLevelOptions
import com.team.smartnutrition.auth.viewmodel.goalOptions
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.ui.text.input.KeyboardType
import com.team.smartnutrition.common.components.ErrorCard
import com.team.smartnutrition.common.components.LoadingScreen
import com.team.smartnutrition.navigation.Screen
```

```
@OptIn(ExperimentalMaterial3Api::class)
```

```
@Composable
```

```
fun ProfileViewScreen(
```

```
    navController: NavController,
```

```
    viewModel: ProfileViewViewModel = viewModel()

```

```
) {
```

```
    val uiState by viewModel.uiState.collectAsState()
```

```
    LaunchedEffect(uiState.isSignedOut) {
```

```
        if (uiState.isSignedOut) {
```

```
            navController.navigate(Screen.Login.route) {
```

```
                popUpTo(0) { inclusive = true }
            }
        }
        viewModel.onSignedOut()
    }
}
```

```
    when {
```

```
        uiState.isLoading -> LoadingScreen()
```

```
        uiState.errorMessage != null && uiState.user == null -> {
```

```
            ErrorCard(message = uiState.errorMessage ?: stringResource(R.string.error_generic), onRetry = { viewModel.loadProfile() })
        }
    }
    uiState.isEditing -> EditProfileContent(uiState, viewModel)
    else -> ViewProfileContent(uiState, viewModel, navController)
}
```

```
}
```

```
@OptIn(ExperimentalMaterial3Api::class)
```

```
@Composable
```

```
private fun ViewProfileContent(
```

```
    uiState: ProfileViewUiState,
```

```

viewModel: ProfileViewViewModel,
navController: NavController
) {
    val user = uiState.user ?: return

    Scaffold(
        topBar = {
            TopAppBar(
                title = { Text(stringResource(R.string.profile_title)) },
                actions = {
                    IconButton(onClick = { viewModel.startEditing() }) {
                        Icon(Icons.Filled.Edit, contentDescription = stringResource(R.string.btn_edit))
                    }
                }
            )
        }
    ) { padding ->
        Column(
            modifier = Modifier.fillMaxSize().padding(padding)
                .verticalScroll(rememberScrollState()).padding(horizontal = 20.dp, vertical = 8.dp),
            horizontalAlignment = Alignment.CenterHorizontally
        ) {
            // Avatar
            Surface(modifier = Modifier.size(80.dp), shape = CircleShape, color = MaterialTheme.colorScheme.primaryContainer) {
                Box(contentAlignment = Alignment.Center) {
                    Text(user.displayName.firstOrNull()?.uppercase() ?: "?",
                        style = MaterialTheme.typography.headlineLarge, color = MaterialTheme.colorScheme.primary)
                }
            }
            Spacer(Modifier.height(12.dp))
            Text(user.displayName, style = MaterialTheme.typography.headlineSmall.copy(fontWeight = FontWeight.Bold))
            Text(user.email, style = MaterialTheme.typography.bodyMedium, color = MaterialTheme.colorScheme.onSurfaceVariant)
            Spacer(Modifier.height(20.dp))

            // Info Card
            Card(modifier = Modifier.fillMaxWidth(), shape = RoundedCornerShape(16.dp),
                colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surfaceVariant)) {
                Column(modifier = Modifier.padding(16.dp)) {
                    Text(stringResource(R.string.info_title), style = MaterialTheme.typography.titleMedium.copy(fontWeight = FontWeight.SemiBold))
                    Spacer(Modifier.height(12.dp))
                    InfoRow(stringResource(R.string.gender_label), if (user.gender == "male") "♂" + stringResource(R.string.gender_male) else "♀")
                    InfoRow(stringResource(R.string.age_label), "${HealthCalculator.calculateAge(user.birthYear)} (${user.birthYear})")
                    InfoRow(stringResource(R.string.height_label), "${user.heightCm} cm")
                    InfoRow(stringResource(R.string.weight_label), "${("%.1f".format(user.weightKg))} kg")
                }
                val goalLabel = when(user.goal) {
                    "lose_weight" -> "🔥" + stringResource(R.string.goal_lose)
                    "gain_muscle" -> "💪" + stringResource(R.string.goal_gain)
                    else -> "🏋️" + stringResource(R.string.goal_maintain)
                }
                InfoRow(stringResource(R.string.goal_label), goalLabel)

                val activityLabel = when(user.activityLevel) {
                    1.2 -> "🧘" + stringResource(R.string.activity_sedentary)
                    1.375 -> "🚶" + stringResource(R.string.activity_light)
                    1.55 -> "🏃" + stringResource(R.string.activity_moderate)
                    1.725 -> "🏊" + stringResource(R.string.activity_very)
                    else -> "🔥" + stringResource(R.string.activity_extreme)
                }
                InfoRow(stringResource(R.string.activity_label), activityLabel)
            }
        }
    }
}

```

```

        Spacer(Modifier.height(16.dp))

    // Health Metrics
    Text("📊 " + stringResource(R.string.health_indices), style = MaterialTheme.typography.titleMedium.copy(fontWeight = FontWeight.SemiB
        modifier = Modifier.fillMaxWidth().padding(bottom = 8.dp))
    Row(Modifier.fillMaxWidth(), horizontalArrangement = Arrangement.spacedBy(8.dp)) {
        MetricCard("BMI", "${"%1f".format(user.bmi)}", HealthCalculator.getBmiCategory(user.bmi), Modifier.weight(1f))
        MetricCard("BMR", "${"%0f".format(user.bmr)}", "kcal", Modifier.weight(1f))
    }
    Spacer(Modifier.height(8.dp))
    Row(Modifier.fillMaxWidth(), horizontalArrangement = Arrangement.spacedBy(8.dp)) {
        MetricCard("TDEE", "${"%0f".format(user.tdee)}", "kcal/ngày", Modifier.weight(1f))
        MetricCard(stringResource(R.string.goal_label), "${user.calorieTarget}", "kcal/ngày", Modifier.weight(1f))
    }
    Spacer(Modifier.height(16.dp))

    // Macros
    Card(Modifier.fillMaxWidth(), shape = RoundedCornerShape(16.dp),
        colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surfaceVariant)) {
        Column(Modifier.padding(16.dp)) {
            Text(stringResource(R.string.daily_macros), style = MaterialTheme.typography.titleMedium.copy(fontWeight = FontWeight.SemiBolt
            Spacer(Modifier.height(12.dp))
            MacroBar("Protein", user.proteinTarget, 200, MaterialTheme.colorScheme.primary)
            Spacer(Modifier.height(8.dp))
            MacroBar("Carbs", user.carbTarget, 400, MaterialTheme.colorScheme.secondary)
            Spacer(Modifier.height(8.dp))
            MacroBar("Fat", user.fatTarget, 120, MaterialTheme.colorScheme.tertiary)
        }
    }
    Spacer(Modifier.height(20.dp))

    // Actions
    OutlinedButton(onClick = { navController.navigate(Screen.WeightLog.route) },
        modifier = Modifier.fillMaxWidth().height(48.dp), shape = RoundedCornerShape(12.dp)) {
        Icon(Icons.Filled.MonitorWeight, null, Modifier.size(18.dp))
        Spacer(Modifier.width(8.dp)); Text(stringResource(R.string.weight_log_btn))
    }
    Spacer(Modifier.height(8.dp))
    OutlinedButton(onClick = { navController.navigate(Screen.Settings.route) },
        modifier = Modifier.fillMaxWidth().height(48.dp), shape = RoundedCornerShape(12.dp)) {
        Icon(Icons.Filled.Settings, null, Modifier.size(18.dp))
        Spacer(Modifier.width(8.dp)); Text(stringResource(R.string.settings_title))
    }
    Spacer(Modifier.height(8.dp))
    TextButton(onClick = { viewModel.signOut() }, modifier = Modifier.fillMaxWidth()) {
        Icon(Icons.Filled.Logout, null, tint = MaterialTheme.colorScheme.error, modifier = Modifier.size(18.dp))
        Spacer(Modifier.width(8.dp)); Text(stringResource(R.string.logout), color = MaterialTheme.colorScheme.error)
    }
    Spacer(Modifier.height(16.dp))
}

@OptIn(ExperimentalMaterial3Api::class)
@Composable
private fun EditProfileContent(uiState: ProfileViewState, viewModel: ProfileViewViewModel) {
    val snackbarHostState = remember { SnackbarHostState() }
    LaunchedEffect(uiState.errorMessage) { uiState.errorMessage?.let { snackbarHostState.showSnackbar(it) } }

    var birthYearInputText by remember { mutableStateOf(uiState.editBirthYear.toString()) }
    LaunchedEffect(uiState.editBirthYear) {
        if (birthYearInputText.toIntOrNull() != uiState.editBirthYear) {

```

```

        birthYearInputText = uiState.editBirthYear.toString()
    }
}

var heightInputText by remember { mutableStateOf(uiState.editHeightCm.toString()) }
LaunchedEffect(uiState.editHeightCm) {
    if (heightInputText.toIntOrNull() != uiState.editHeightCm) {
        heightInputText = uiState.editHeightCm.toString()
    }
}

var weightInputText by remember { mutableStateOf("%.1f".format(uiState.editWeightKg)) }
LaunchedEffect(uiState.editWeightKg) {
    val currentFloat = weightInputText.toDoubleOrNull() ?: 0.0
    if (Math.abs(currentFloat - uiState.editWeightKg) > 0.05) {
        weightInputText = "%.1f".format(uiState.editWeightKg)
    }
}

Scaffold(
    topBar = {
        TopAppBar(title = { Text(stringResource(R.string.edit_profile_title)) },
            navigationIcon = { IconButton(onClick = { viewModel.cancelEditing() }) { Icon(Icons.Filled.Close, stringResource(R.string.cancel)) } },
            actions = {
                TextButton(onClick = { viewModel.saveChanges() }, enabled = !uiState.isSaving) {
                    if (uiState.isSaving) CircularProgressIndicator(Modifier.size(16.dp), strokeWidth = 2.dp)
                    else Text(stringResource(R.string.save), fontWeight = FontWeight.Bold)
                }
            })
    },
    snackbarHost = { SnackbarHost(snackbarHostState) }
) { padding ->
    Column(Modifier.fillMaxSize().padding(padding).verticalScroll(rememberScrollState()).padding(24.dp),
        verticalArrangement = Arrangement.spacedBy(16.dp)) {
        OutlinedTextField(value = uiState.editDisplayName, onValueChange = { viewModel.updateEditDisplayName(it) },
            label = { Text(stringResource(R.string.display_name_label)) }, leadingIcon = { Icon(Icons.Filled.Person, null) },
            shape = RoundedCornerShape(12.dp), modifier = Modifier.fillMaxWidth(), singleLine = true)

        Text(stringResource(R.string.gender_label), style = MaterialTheme.typography.titleMedium)
        Row(horizontalArrangement = Arrangement.spacedBy(12.dp)) {
            FilterChip(selected = uiState.editGender == "male", onClick = { viewModel.updateEditGender("male") }, label = { Text("♂ " + stringResource(R.string.male_label)) })
            FilterChip(selected = uiState.editGender == "female", onClick = { viewModel.updateEditGender("female") }, label = { Text("♀ " + stringResource(R.string.female_label)) })
        }

        Text(stringResource(R.string.birth_year_label, uiState.editBirthYear), style = MaterialTheme.typography.titleMedium)
        Slider(value = uiState.editBirthYear.toFloat(), onValueChange = { viewModel.updateEditBirthYear(it.toInt()) }, valueRange = 1940f..2015f)
        OutlinedTextField(
            value = birthYearInputText,
            onValueChange = { newValue ->
                if (newValue.isEmpty() || newValue.all { it.isDigit() }) {
                    birthYearInputText = newValue
                    newValue.toIntOrNull()?.let { year ->
                        if (year in 1940..2015) {
                            viewModel.updateEditBirthYear(year)
                        }
                    }
                }
            })
        },
        label = { Text(stringResource(R.string.birth_year_label, uiState.editBirthYear).substringBefore(":") + " (1940 - 2015)") },
        keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Number),
        shape = RoundedCornerShape(12.dp),
        modifier = Modifier.fillMaxWidth(),
    
```

```

        singleLine = true
    )

    Text(stringResource(R.string.height_slider_label, uiState.editHeightCm), style = MaterialTheme.typography.titleMedium)
    Slider(value = uiState.editHeightCm.toFloat(), onValueChange = { viewModel.updateEditHeightCm(it.toInt()) }, valueRange = 100f..250f)
    OutlinedTextField(
        value = heightInputText,
        onValueChange = { newValue ->
            if (newValue.isEmpty() || newValue.all { it.isDigit() }) {
                heightInputText = newValue
                newValue.toIntOrNull()?.let { height ->
                    if (height in 100..250) {
                        viewModel.updateEditHeightCm(height)
                    }
                }
            }
        },
        label = { Text("Nhập chiều cao thủ công") },
        suffix = { Text("cm") },
        keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Number),
        shape = RoundedCornerShape(12.dp),
        modifier = Modifier.fillMaxWidth(),
        singleLine = true
    )

```

```

    Text(stringResource(R.string.weight_slider_label, uiState.editWeightKg), style = MaterialTheme.typography.titleMedium)
    Slider(value = uiState.editWeightKg.toFloat(), onValueChange = { viewModel.updateEditWeightKg((it * 10).toInt() / 10.0) }, valueRange = 30f..250f)
    OutlinedTextField(
        value = weightInputText,
        onValueChange = { newValue ->
            val normalized = newValue.replace(',', '.')
            if (normalized.isEmpty() || normalized.toDoubleOrNull() != null || normalized == "." || normalized.endsWith(".")) {
                weightInputText = newValue
                normalized.toDoubleOrNull()?.let { weight ->
                    if (weight >= 30.0 && weight <= 200.0) {
                        viewModel.updateEditWeightKg(weight)
                    }
                }
            }
        },
        label = { Text("Nhập cân nặng thủ công") },
        suffix = { Text("kg") },
        keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Decimal),
        shape = RoundedCornerShape(12.dp),
        modifier = Modifier.fillMaxWidth(),
        singleLine = true
    )

```

```

    Text(stringResource(R.string.goal_label), style = MaterialTheme.typography.titleMedium)
    goalOptions.forEach { option ->
        FilterChip(selected = uiState.editGoal == option.value, onClick = { viewModel.updateEditGoal(option.value) },
            label = {
                val labelText = when (option.value) {
                    "lose_weight" -> "🔥 " + stringResource(R.string.goal_lose)
                    "gain_muscle" -> "💪 " + stringResource(R.string.goal_gain)
                    else -> "👤 " + stringResource(R.string.goal_maintain)
                }
                Text(labelText)
            }
        })
    }

```

```

    Text(stringResource(R.string.activity_label), style = MaterialTheme.typography.titleMedium)

```



```

        activityLevelOptions.forEach { option ->
            Row(verticalAlignment = Alignment.CenterVertically) {
                RadioButton(selected = uiState.editActivityLevel == option.value, onClick = { viewModel.updateEditActivityLevel(option.value)
                    val labelText = when (option.value) {
                        1.2 -> "🧘" + stringResource(R.string.activity_sedentary)
                        1.375 -> "🚶" + stringResource(R.string.activity_light)
                        1.55 -> "🚴" + stringResource(R.string.activity_moderate)
                        1.725 -> "🏊" + stringResource(R.string.activity_very)
                        else -> "🔥" + stringResource(R.string.activity_extreme)
                    }
                }
                Text(labelText)
            }
        }
    }
}

```

@Composable

```

private fun InfoRow(label: String, value: String) {
    Row(Modifier.fillMaxWidth().padding(vertical = 4.dp), horizontalArrangement = Arrangement.SpaceBetween) {
        Text(label, style = MaterialTheme.typography.bodyMedium, color = MaterialTheme.colorScheme.onSurfaceVariant)
        Text(value, style = MaterialTheme.typography.bodyMedium.copy(fontWeight = FontWeight.Medium))
    }
}

```

@Composable

```

private fun MetricCard(label: String, value: String, unit: String, modifier: Modifier = Modifier) {
    Card(modifier = modifier, shape = RoundedCornerShape(12.dp),
        colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.primaryContainer)) {
        Column(Modifier.fillMaxWidth().padding(12.dp), horizontalAlignment = Alignment.CenterHorizontally) {
            Text(label, style = MaterialTheme.typography.bodySmall, color = MaterialTheme.colorScheme.onPrimaryContainer.copy(alpha = 0.7f))
            Spacer(Modifier.height(4.dp))
            Text(value, style = MaterialTheme.typography.titleLarge.copy(fontWeight = FontWeight.Bold), color = MaterialTheme.colorScheme.primary)
            if (unit.isNotBlank()) Text(unit, style = MaterialTheme.typography.bodySmall, color = MaterialTheme.colorScheme.onPrimaryContainer.co
        }
    }
}

```

@Composable

```

private fun MacroBar(label: String, value: Int, maxValue: Int, color: androidx.compose.ui.graphics.Color) {
    Column {
        Row(Modifier.fillMaxWidth(), horizontalArrangement = Arrangement.SpaceBetween) {
            Text(label, style = MaterialTheme.typography.bodyMedium)
            Text("${value}g", style = MaterialTheme.typography.bodyMedium.copy(fontWeight = FontWeight.Medium))
        }
        Spacer(Modifier.height(4.dp))
        LinearProgressIndicator(
            progress = { if (maxValue > 0) (value.toFloat() / maxValue).coerceIn(0f, 1f) else 0f },
            modifier = Modifier.fillMaxWidth().height(8.dp).clip(RoundedCornerShape(4.dp)),
            color = color, trackColor = color.copy(alpha = 0.2f))
    }
}

```

File: auth/RegisterScreen.kt

```

package com.team.smartnutrition.auth

import androidx.compose.foundation.layout.*
import androidx.compose.foundation.rememberScrollState
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.text.KeyboardActions
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.foundation.verticalScroll
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Email
import androidx.compose.material.icons.filled.Lock
import androidx.compose.material.icons.filled.Visibility
import androidx.compose.material.icons.filled.VisibilityOff
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.focus.FocusDirection
import androidx.compose.ui.platform.LocalFocusManager
import androidx.compose.ui.res.stringResource
import androidx.compose.ui.text.input.ImeAction
import androidx.compose.ui.text.input.KeyboardType
import androidx.compose.ui.text.input.PasswordVisualTransformation
import androidx.compose.ui.text.input.VisualTransformation
import androidx.compose.ui.unit.dp
import androidx.lifecycle.viewmodel.compose.viewModel
import androidx.navigation.NavController
import com.team.smartnutrition.R
import com.team.smartnutrition.auth.viewmodel.RegisterViewModel
import com.team.smartnutrition.common.components.SmartTopBar
import com.team.smartnutrition.navigation.Screen

/**
 * =====
 * MODULE 1 - TV1: MÀN HÌNH ĐĂNG KÝ
 * =====
 *
 * Tính năng:
 * - Form đăng ký (Email, Password, Confirm Password)
 * - Validation realtime từng field
 * - Firebase createUserWithEmailAndPassword
 * - Thành công → navigate ProfileSetup
 */
@Composable
fun RegisterScreen(
    navController: NavController,
    viewModel: RegisterViewModel = viewModel()
) {
    val uiState by viewModel.uiState.collectAsState()
    var passwordVisible by remember { mutableStateOf(false) }
    var confirmPasswordVisible by remember { mutableStateOf(false) }
    val focusManager = LocalFocusManager.current
    val snackbarHostState = remember { SnackbarHostState() }

    // Handle navigation after successful registration
    LaunchedEffect(uiState.isRegistered) {
        if (uiState.isRegistered) {
            navController.navigate(Screen.ProfileSetup.route) {
                popUpTo(Screen.Login.route) { inclusive = true }
            }
            viewModel.onNavigated()
        }
    }
}

```

```

    }

    // Handle error snackbar
    LaunchedEffect(uiState.errorMessage) {
        uiState.errorMessage?.let { message ->
            snackbarHostState.showSnackbar(message)
        }
    }
}

Scaffold(
    topBar = {
        SmartTopBar(
            title = stringResource(R.string.register_title),
            onBackPressed = { navController.popBackStack() }
        )
    },
    snackbarHost = { SnackbarHost(snackbarHostState) }
) { padding ->
    Column(
        modifier = Modifier
            .fillMaxSize()
            .padding(padding)
            .padding(24.dp)
            .verticalScroll(rememberScrollState()),
        horizontalAlignment = Alignment.CenterHorizontally,
        verticalArrangement = Arrangement.Center
    ) {
        Spacer(modifier = Modifier.weight(1f))

        Text(
            text = stringResource(R.string.create_account_title),
            style = MaterialTheme.typography.headlineMedium
        )
        Text(
            text = stringResource(R.string.register_subtitle),
            style = MaterialTheme.typography.bodyMedium,
            color = MaterialTheme.colorScheme.onSurfaceVariant,
            modifier = Modifier.padding(top = 4.dp, bottom = 32.dp)
        )

        // == Email Field ==
        OutlinedTextField(
            value = uiState.email,
            onValueChange = { viewModel.updateEmail(it) },
            label = { Text(stringResource(R.string.email_label)) },
            leadingIcon = {
                Icon(Icons.Filled.Email, contentDescription = null)
            },
            keyboardOptions = KeyboardOptions(
                keyboardType = KeyboardType.Email,
                imeAction = ImeAction.Next
            ),
            keyboardActions = KeyboardActions(
                onNext = { focusManager.moveFocus(FocusDirection.Down) }
            ),
            shape = RoundedCornerShape(12.dp),
            modifier = Modifier.fillMaxWidth(),
            singleLine = true,
            isError = uiState.emailError != null,
            supportingText = uiState.emailError?.let {
                { Text(it, color = MaterialTheme.colorScheme.error) }
            },
        ),
    }
}

```

```

        enabled = !uiState.isLoading
    )
    Spacer(modifier = Modifier.height(8.dp))

    // == Password Field ==
    OutlinedTextField(
        value = uiState.password,
        onChange = { viewModel.updatePassword(it) },
        label = { Text(stringResource(R.string.password_label)) },
        leadingIcon = {
            Icon(Icons.Filled.Lock, contentDescription = null)
        },
        trailingIcon = {
            IconButton(onClick = { passwordVisible = !passwordVisible }) {
                Icon(
                    if (passwordVisible) Icons.Filled.Visibility
                    else Icons.Filled.VisibilityOff,
                    contentDescription = stringResource(R.string.password_label)
                )
            }
        },
        visualTransformation = if (passwordVisible) VisualTransformation.None
        else PasswordVisualTransformation(),
        keyboardOptions = KeyboardOptions(
            keyboardType = KeyboardType.Password,
            imeAction = ImeAction.Next
        ),
        keyboardActions = KeyboardActions(
            onNext = { focusManager.moveFocus(FocusDirection.Down) }
        ),
        shape = RoundedCornerShape(12.dp),
        modifier = Modifier.fillMaxWidth(),
        singleline = true,
        isError = uiState.passwordError != null,
        supportingText = {
            if (uiState.passwordError != null) {
                Text(uiState.passwordError!!, color = MaterialTheme.colorScheme.error)
            } else if (uiState.password.isNotBlank()) {
                Text(stringResource(R.string.password_valid), color = MaterialTheme.colorScheme.primary)
            } else {
                Text(
                    stringResource(R.string.password_requirement),
                    color = MaterialTheme.colorScheme.onSurfaceVariant
                )
            }
        },
        enabled = !uiState.isLoading
    )
    Spacer(modifier = Modifier.height(8.dp))

    // == Confirm Password Field ==
    OutlinedTextField(
        value = uiState.confirmPassword,
        onChange = { viewModel.updateConfirmPassword(it) },
        label = { Text(stringResource(R.string.confirm_password_label)) },
        leadingIcon = {
            Icon(Icons.Filled.Lock, contentDescription = null)
        },
        trailingIcon = {
            IconButton(onClick = { confirmPasswordVisible = !confirmPasswordVisible }) {
                Icon(
                    if (confirmPasswordVisible) Icons.Filled.Visibility

```

```

        else Icons.Filled.VisibilityOff,
        contentDescription = stringResource(R.string.confirm_password_label)
    )
    },
    visualTransformation = if (confirmPasswordVisible) VisualTransformation.None
    else PasswordVisualTransformation(),
    keyboardOptions = KeyboardOptions(
        keyboardType = KeyboardType.Password,
        imeAction = ImeAction.Done
    ),
    keyboardActions = KeyboardActions(
        onDone = {
            focusManager.clearFocus()
            if (viewModel.isFormValid()) viewModel.register()
        }
    ),
    shape = RoundedCornerShape(12.dp),
    modifier = Modifier.fillMaxWidth(),
    singleLine = true,
    isError = uiState.confirmPasswordError != null,
    supportingText = uiState.confirmPasswordError?.let {
        { Text(it, color = MaterialTheme.colorScheme.error) }
    },
    enabled = !uiState.isLoading
)
Spacer(modifier = Modifier.height(24.dp))

// == Register Button ==
Button(
    onClick = { viewModel.register() },
    modifier = Modifier
        .fillMaxWidth()
        .height(52.dp),
    shape = RoundedCornerShape(12.dp),
    enabled = !uiState.isLoading && viewModel.isFormValid()
) {
    if (uiState.isLoading) {
        CircularProgressIndicator(
            modifier = Modifier.size(20.dp),
            color = MaterialTheme.colorScheme.onPrimary,
            strokeWidth = 2.dp
        )
    } else {
        Text(stringResource(R.string.register_button), style = MaterialTheme.typography.labelLarge)
    }
}

Spacer(modifier = Modifier.weight(1f))
}
}
}

```

File: auth/WeightLogScreen.kt

```

package com.team.smartnutrition.auth

import androidx.compose.foundation.layout.*
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.foundation.text.KeyboardActions
import androidx.compose.foundation.text.KeyboardOptions
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.MonitorWeight
import androidx.compose.material.icons.filled.Save
import androidx.compose.material3.*
import androidx.compose.runtime.*
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.platform.LocalFocusManager
import androidx.compose.ui.res.stringResource
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.text.input.ImeAction
import androidx.compose.ui.text.input.KeyboardType
import androidx.compose.ui.unit.dp
import androidx.lifecycle.viewmodel.compose.viewModel
import androidx.navigation.NavController
import com.team.smartnutrition.R
import com.team.smartnutrition.auth.util.HealthCalculator
import com.team.smartnutrition.auth.viewmodel.WeightLogViewModel
import com.team.smartnutrition.common.components.SmartTopBar
import java.time.LocalDate
import java.time.format.DateTimeFormatter

/**
 * =====
 * MODULE 1 - TV1: NHẬT KÝ CÂN NẶNG
 * =====
 *
 * Tính năng:
 * - Nhập cân nặng hôm nay + tính BMI realtime
 * - Lưu vào weightLog/{date} + update user profile
 * - Hiện thị lịch sử cân nặng (LazyColumn DESC)
 */
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun WeightLogScreen(
    navController: NavController,
    viewModel: WeightLogViewModel = viewModel()
) {
    val uiState by viewModel.uiState.collectAsState()
    val focusManager = LocalFocusManager.current
    val snackbarHostState = remember { SnackbarHostState() }
    val displayFormatter = DateTimeFormatter.ofPattern("dd/MM/yyyy")

    // Handle messages
    val successMsg = stringResource(R.string.weight_logged_success)
    LaunchedEffect(uiState.errorMessage) {
        uiState.errorMessage?.let {
            snackbarHostState.showSnackbar(it)
            viewModel.clearError()
        }
    }
    LaunchedEffect(uiState.saveSuccess) {
        if (uiState.saveSuccess) {
            snackbarHostState.showSnackbar(successMsg)
        }
    }
}

```

```

        viewModel.clearSaveSuccess()
    }
}

Scaffold(
    topBar = {
        SmartTopBar(
            title = stringResource(R.string.weight_log_btn),
            onBackPressed = { navController.popBackStack() }
        )
    },
    snackbarHost = { SnackbarHost(snackbarHostState) }
) { padding ->
    if (uiState.isLoading) {
        Box(Modifier.fillMaxSize().padding(padding), contentAlignment = Alignment.Center) {
            CircularProgressIndicator()
        }
    } else {
        LazyColumn(
            modifier = Modifier.fillMaxSize().padding(padding),
            contentPadding = PaddingValues(20.dp),
            verticalArrangement = Arrangement.spacedBy(12.dp)
        ) {
            // == Input Card ==
            item {
                Card(
                    modifier = Modifier.fillMaxWidth(),
                    shape = RoundedCornerShape(16.dp),
                    colors = CardDefaults.cardColors(
                        containerColor = MaterialTheme.colorScheme.primaryContainer
                    )
                ) {
                    Column(Modifier.padding(20.dp)) {
                        Text(stringResource(R.string.today_weight_title),
                            style = MaterialTheme.typography.titleMedium.copy(fontWeight = FontWeight.Bold),
                            color = MaterialTheme.colorScheme.onPrimaryContainer)
                        Spacer(Modifier.height(12.dp))

                        OutlinedTextField(
                            value = uiState.currentWeight,
                            onValueChange = { viewModel.updateWeight(it) },
                            label = { Text(stringResource(R.string.weight_label)) },
                            leadingIcon = { Icon(Icons.Filled.MonitorWeight, null) },
                            suffix = { Text("kg") },
                            keyboardOptions = KeyboardOptions(
                                keyboardType = KeyboardType.Decimal,
                                imeAction = ImeAction.Done
                            ),
                            keyboardActions = KeyboardActions(onDone = {
                                focusManager.clearFocus()
                                viewModel.saveWeight()
                            }),
                            shape = RoundedCornerShape(12.dp),
                            modifier = Modifier.fillMaxWidth(),
                            singleLine = true,
                            colors = OutlinedTextFieldDefaults.colors(
                                unfocusedContainerColor = MaterialTheme.colorScheme.surface,
                                focusedContainerColor = MaterialTheme.colorScheme.surface
                            )
                        )
                    }
                }
            }

            // BMI Preview

```

```

                Text(formattedDate, style = MaterialTheme.typography.bodyMedium,
                    color = MaterialTheme.colorScheme.onSurfaceVariant)
                Text("BMI: ${"%0.2f".format(entry.bmi)}",
                    style = MaterialTheme.typography.bodySmall,
                    color = MaterialTheme.colorScheme.onSurfaceVariant)
            }
            Text("${"%0.1f".format(entry.weightKg)} kg",

```



```

                style = MaterialTheme.typography.titleMedium.copy(fontWeight = FontWeight.Bold),
                color = MaterialTheme.colorScheme.primary)
            }
        }
    }

    // == Empty State ==
    if (uiState.history.isEmpty()) {
        item {
            Card(Modifier.fillMaxWidth(), shape = RoundedCornerShape(12.dp),
                colors = CardDefaults.cardColors(containerColor = MaterialTheme.colorScheme.surfaceVariant)) {
                Column(Modifier.fillMaxWidth().padding(32.dp),
                    horizontalAlignment = Alignment.CenterHorizontally) {
                    Text("📊", style = MaterialTheme.typography.displaySmall)
                    Spacer(Modifier.height(8.dp))
                    Text(stringResource(R.string.no_weight_data), style = MaterialTheme.typography.bodyMedium,
                        color = MaterialTheme.colorScheme.onSurfaceVariant)
                    Text(stringResource(R.string.start_weight_log_hint),
                        style = MaterialTheme.typography.bodySmall,
                        color = MaterialTheme.colorScheme.onSurfaceVariant)
                }
            }
        }
    }
}

```

File: auth/data/UserRepository.kt

```

package com.team.smartnutrition.auth.data

import com.google.firebase.Timestamp
import com.google.firebase.auth.AuthCredential
import com.google.firebase.auth.FirebaseAuth
import com.google.firebase.auth.FirebaseUser
import com.google.firebase.firestore.FirebaseFirestore
import com.google.firebase.firestore.Query
import com.team.smartnutrition.auth.model.User
import com.team.smartnutrition.auth.model.WeightEntry
import kotlinx.coroutines.channels.awaitClose
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.callbackFlow
import kotlinx.coroutines.tasks.await

/**
 * =====
 * USER REPOSITORY - Lớp dữ liệu duy nhất
 * =====
 *
 * Đóng gói toàn bộ Firebase Auth + Firestore operations.
 * Tất cả ViewModel dùng chung repository này.
 */
class UserRepository {

    private val auth = FirebaseAuth.getInstance()
    private val firestore = FirebaseFirestore.getInstance()

    // == FIREBASE AUTH ==

    /** User hiện tại (null nếu chưa login) */
    val currentUser: FirebaseUser?
        get() = auth.currentUser

    /** UID của user hiện tại */
    val currentUid: String?
        get() = auth.currentUser?.uid

    /**
     * Đăng nhập bằng Email/Password.
     * @throws Exception nếu thất bại
     */
    suspend fun signInWithEmail(email: String, password: String): FirebaseUser {
        val result = kotlinx.coroutines.withTimeoutOrNull(15000) {
            auth.signInWithEmailAndPassword(email, password).await()
        } ?: throw Exception("timeout")
        return result.user ?: throw Exception("Đăng nhập thất bại")
    }

    /**
     * Đăng nhập bằng Google Credential.
     * @param credential AuthCredential từ Google Sign-In flow
     */
    suspend fun signInWithCredential(credential: AuthCredential): FirebaseUser {
        val result = kotlinx.coroutines.withTimeoutOrNull(15000) {
            auth.signInWithCredential(credential).await()
        } ?: throw Exception("timeout")
        return result.user ?: throw Exception("Đăng nhập Google thất bại")
    }

    /**
     * Đăng ký tài khoản mới bằng Email/Password.

```

```

    * @throws Exception nếu email đã tồn tại, password yếu, v.v.
    */
    suspend fun registerWithEmail(email: String, password: String): FirebaseUser {
        val result = kotlinx.coroutines.withTimeoutOrNull(15000) {
            auth.createUserWithEmailAndPassword(email, password).await()
        } ?: throw Exception("timeout")
        return result.user ?: throw Exception("Đăng ký thất bại")
    }

    /** Đăng xuất */
    fun signOut() {
        auth.signOut()
    }

    // == FIRESTORE: USER PROFILE ==

    /**
     * Kiểm tra user đã có profile trên Firestore chưa.
     * Dùng khi login để quyết định navigate ProfileSetup hay Home.
     */
    suspend fun hasProfile(uid: String): Boolean {
        return try {
            val doc = kotlinx.coroutines.withTimeout(3000) {
                firestore.collection("users").document(uid).get().await()
            }
            doc.exists()
        } catch (e: Exception) {
            // Fallback đọc từ cache local
            val doc = firestore.collection("users").document(uid)
                .get(com.google.firebase.firestore.Source.CACHE).await()
            doc.exists()
        }
    }

    /**
     * Lấy profile user từ Firestore.
     * @return User hoặc null nếu chưa có
     */
    suspend fun getUser(uid: String): User? {
        val doc = try {
            kotlinx.coroutines.withTimeout(3000) {
                firestore.collection("users").document(uid).get().await()
            }
        } catch (e: Exception) {
            // Fallback đọc từ cache local
            firestore.collection("users").document(uid)
                .get(com.google.firebase.firestore.Source.CACHE).await()
        }
        return doc.toObject(User::class.java)?.copy(uid = uid)
    }

    /**
     * Lưu/cập nhật profile user lên Firestore.
     * Dùng cho cả ProfileSetup (lần đầu) và Profile Edit.
     */
    suspend fun saveUser(user: User) {
        val data = hashMapOf(
            "email" to user.email,
            "displayName" to user.displayName,
            "gender" to user.gender,
            "birthYear" to user.birthYear,
            "heightCm" to user.heightCm,

```

```

        "weightKg" to user.weightKg,
        "activityLevel" to user.activityLevel,
        "goal" to user.goal,
        "bmi" to user.bmi,
        "bmr" to user.bmr,
        "tdee" to user.tdee,
        "proteinTarget" to user.proteinTarget,
        "carbTarget" to user.carbTarget,
        "fatTarget" to user.fatTarget,
        "calorieTarget" to user.calorieTarget,
        "updatedAt" to Timestamp.now()
    )
    // Chỉ set createdAt khi tạo mới
    if (user.createdAt == null) {
        data["createdAt"] = Timestamp.now()
    }

    // Ghi bất đồng bộ (offline-first) - ghi lập tức vào SQLite cache
    firestore.collection("users").document(user.uid)
        .set(data, com.google.firebase.firestore.SetOptions.merge())
}

// == FIRESTORE: WEIGHT LOG ==

/**
 * Lưu entry cân nặng ngày hôm nay.
 * Document ID = date → mỗi ngày chỉ 1 record (upsert).
 * Đồng thời cập nhật weightKg + bmi trong user profile.
 */
suspend fun logWeight(uid: String, date: String, weightKg: Double, bmi: Double) {
    val entry = hashMapOf(
        "weightKg" to weightKg,
        "bmi" to bmi,
        "loggedAt" to Timestamp.now()
    )

    // Batch write: cập nhật cả weightLog và user profile cùng lúc bất đồng bộ
    val batch = firestore.batch()

    // 1. Upsert weightLog/{date}
    val weightRef = firestore.collection("users").document(uid)
        .collection("weightLog").document(date)
    batch.set(weightRef, entry)

    // 2. Update user profile weightKg + bmi
    val userRef = firestore.collection("users").document(uid)
    batch.update(userRef, mapOf(
        "weightKg" to weightKg,
        "bmi" to bmi,
        "updatedAt" to Timestamp.now()
    ))

    // Ghi bất đồng bộ (offline-first)
    batch.commit()
}

/**
 * Lấy lịch sử cân nặng, sắp xếp mới nhất trước.
 * @param limit số lượng tối đa entries trả về
 */
suspend fun getWeightHistory(uid: String, limit: Long = 30): List<WeightEntry> {
    val snapshot = try {

```

```
kotlinx.coroutines.withTimeout(3000) {  
    firestore.collection("users").document(uid)  
        .collection("weightLog")  
        .orderBy("loggedAt", Query.Direction.DESENDING)  
        .limit(limit)  
        .get()  
        .await()  
}  
} catch (e: Exception) {  
    // Fallback đọc từ cache local  
    firestore.collection("users").document(uid)  
        .collection("weightLog")  
        .orderBy("loggedAt", Query.Direction.DESENDING)  
        .limit(limit)  
        .get(com.google.firebase.firestore.Source.CACHE)  
        .await()  
}  
  
return snapshot.documents.map { doc ->  
    doc.toObject(WeightEntry::class.java)?.copy(date = doc.id)  
        ?: WeightEntry(date = doc.id)  
    }  
}  
}
```

File: auth/model/User.kt

```

package com.team.smartnutrition.auth.model

import com.google.firebase.Timestamp

/**
 * =====
 * DATA MODELS cho Module 1 - Auth & Profile
 * =====
 *
 * Ảnh xạ trực tiếp tới Firestore document:
 * - User → users/{uid}
 * - WeightEntry → users/{uid}/weightLog/{date}
 */

/**
 * User profile data, khớp 1:1 với Firestore schema.
 * Dùng default values để Firestore.toObject() hoạt động.
 */
data class User(
    val uid: String = "",
    val email: String = "",
    val displayName: String = "",
    val gender: String = "male", // "male" | "female"
    val birthYear: Int = 2000,
    val heightCm: Int = 170,
    val weightKg: Double = 65.0,
    val activityLevel: Double = 1.55, // 1.2 → 1.9
    val goal: String = "maintain", // "lose_weight" | "maintain" | "gain_muscle"
    val bmi: Double = 0.0,
    val bmr: Double = 0.0,
    val tdee: Double = 0.0,
    val proteinTarget: Int = 0, // gram/ngày
    val carbTarget: Int = 0, // gram/ngày
    val fatTarget: Int = 0, // gram/ngày
    val calorieTarget: Int = 0, // kcal/ngày
    val createdAt: Timestamp? = null,
    val updatedAt: Timestamp? = null
)

/**
 * Một entry cân nặng trong sub-collection weightLog.
 * Document ID = "yyyy-MM-dd" (mỗi ngày 1 record duy nhất).
 */
data class WeightEntry(
    val weightKg: Double = 0.0,
    val bmi: Double = 0.0,
    val loggedAt: Timestamp? = null,
    val date: String = "" // Document ID, format "yyyy-MM-dd"
)

```

File: auth/util/HealthCalculator.kt

```

package com.team.smartnutrition.auth.util

import java.time.Year
import kotlin.math.pow
import kotlin.math.roundToInt

/**
 * =====
 * HEALTH CALCULATOR - Tính chỉ số sức khỏe
 * =====
 *
 * Pure functions, chạy LOCAL trên app.
 * Công thức Harris-Benedict cho BMR.
 */
object HealthCalculator {

    /**
     * Tính BMI = cân nặng (kg) / chiều cao (m)2
     */
    fun calculateBmi(weightKg: Double, heightCm: Int): Double {
        if (heightCm <= 0) return 0.0
        val heightM = heightCm / 100.0
        return (weightKg / heightM.pow(2)).roundTo(2)
    }

    /**
     * Phân loại BMI theo chuẩn WHO.
     */
    fun getBmiCategory(bmi: Double): String = when {
        bmi < 18.5 -> "Thiếu cân"
        bmi < 25.0 -> "Bình thường"
        bmi < 30.0 -> "Thừa cân"
        else -> "Béo phì"
    }

    /**
     * Tính BMR bằng công thức Harris-Benedict.
     * @param gender "male" hoặc "female"
     * @param age tuổi (năm)
     */
    fun calculateBmr(
        weightKg: Double,
        heightCm: Int,
        age: Int,
        gender: String
    ): Double {
        return if (gender == "male") {
            88.362 + (13.397 * weightKg) + (4.799 * heightCm) - (5.677 * age)
        } else {
            447.593 + (9.247 * weightKg) + (3.098 * heightCm) - (4.330 * age)
        }.roundTo(1)
    }

    /**
     * Tính TDEE = BMR x hệ số vận động.
     * @param activityLevel 1.2 (ít) → 1.9 (rất nhiều)
     */
    fun calculateTdee(bmr: Double, activityLevel: Double): Double {
        return (bmr * activityLevel).roundTo(1)
    }

    /**

```

```

    * Calorie mục tiêu, điều chỉnh theo goal.
    * - lose_weight: TDEE - 500
    * - gain_muscle: TDEE + 300
    * - maintain: TDEE giữ nguyên
    */
    fun calculateCalorieTarget(tdee: Double, goal: String): Int {
        return when (goal) {
            "lose_weight" -> (tdee - 500).roundToInt()
            "gain_muscle" -> (tdee + 300).roundToInt()
            else -> tdee.roundToInt()
        }
    }
}

/**
 * Tính Macros mục tiêu (Protein, Carb, Fat) theo gram/ngày.
 * @return Triple(proteinGrams, carbGrams, fatGrams)
 */
fun calculateMacros(calorieTarget: Int, goal: String): Triple<Int, Int, Int> {
    val cal = calorieTarget.toDouble()
    return when (goal) {
        "gain_muscle" -> Triple(
            (cal * 0.30 / 4).roundToInt(), // 30% Protein (1g = 4 kcal)
            (cal * 0.45 / 4).roundToInt(), // 45% Carb
            (cal * 0.25 / 9).roundToInt() // 25% Fat (1g = 9 kcal)
        )
        "lose_weight" -> Triple(
            (cal * 0.35 / 4).roundToInt(), // 35% Protein
            (cal * 0.35 / 4).roundToInt(), // 35% Carb
            (cal * 0.30 / 9).roundToInt() // 30% Fat
        )
        else -> Triple(
            (cal * 0.25 / 4).roundToInt(), // 25% Protein
            (cal * 0.50 / 4).roundToInt(), // 50% Carb
            (cal * 0.25 / 9).roundToInt() // 25% Fat
        )
    }
}

/**
 * Tính tuổi từ năm sinh.
 */
fun calculateAge(birthYear: Int): Int {
    return Year.now().value - birthYear
}

```

```

/**
 * Tính tất cả chỉ số cùng lúc.
 * Tiện cho ProfileSetup khi cần hiện toàn bộ kết quả.
 */
fun calculateAllMetrics(
    weightKg: Double,
    heightCm: Int,
    birthYear: Int,
    gender: String,
    activityLevel: Double,
    goal: String
): HealthMetrics {
    val age = calculateAge(birthYear)
    val bmi = calculateBmi(weightKg, heightCm)
    val bmr = calculateBmr(weightKg, heightCm, age, gender)
    val tdee = calculateTdee(bmr, activityLevel)
    val calorieTarget = calculateCalorieTarget(tdee, goal)
}

```



```
        val (protein, carb, fat) = calculateMacros(calorieTarget, goal)

        return HealthMetrics(
            bmi = bmi,
            bmiCategory = getBmiCategory(bmi),
            bmr = bmr,
            tdee = tdee,
            calorieTarget = calorieTarget,
            proteinTarget = protein,
            carbTarget = carb,
            fatTarget = fat
        )
    }

    /** Round Double tới N chữ số thập phân */
    private fun Double.roundTo(decimals: Int): Double {
        val factor = 10.0.pow(decimals)
        return (this * factor).roundToInt() / factor
    }
}

/**
 * Kết quả tính toán tất cả chỉ số sức khỏe.
 */
data class HealthMetrics(
    val bmi: Double = 0.0,
    val bmiCategory: String = "",
    val bmr: Double = 0.0,
    val tdee: Double = 0.0,
    val calorieTarget: Int = 0,
    val proteinTarget: Int = 0,
    val carbTarget: Int = 0,
    val fatTarget: Int = 0
)
```

File: auth/util/Validators.kt

```
package com.team.smartnutrition.auth.util

import android.util.Patterns

/**
 * =====
 * VALIDATION UTILS - Module 1
 * =====
 *
 * Quy tắc:
 * - Email: Patterns.EMAIL_ADDRESS regex chuẩn Android
 * - Password: ≥6 ký tự + 1 chữ hoa + 1 chữ số
 * - Confirm: phải trùng khớp hoàn toàn
 */
object Validators {

    /**
     * Validate email theo format chuẩn Android.
     */
    fun isValidEmail(email: String): Boolean {
        return email.isNotBlank() && Patterns.EMAIL_ADDRESS.matcher(email).matches()
    }

    /**
     * Validate password theo quy tắc bảo mật.
     * ≥6 ký tự, ít nhất 1 chữ hoa, ít nhất 1 chữ số.
     */
    fun isValidPassword(password: String): Boolean {
        return password.length >= 6 &&
            password.any { it.isUpperCase() } &&
            password.any { it.isDigit() }
    }

    /**
     * Kiểm tra confirm password trùng khớp.
     */
    fun doPasswordsMatch(password: String, confirmPassword: String): Boolean {
        return password.isNotEmpty() && password == confirmPassword
    }

    /**
     * Lấy thông báo lỗi email (null = hợp lệ).
     */
    fun getEmailError(email: String): String? {
        return when {
            email.isBlank() -> null // Chưa nhập, không hiện lỗi
            !Patterns.EMAIL_ADDRESS.matcher(email).matches() -> "Email không hợp lệ"
            else -> null
        }
    }

    /**
     * Lấy thông báo lỗi password (null = hợp lệ).
     * Trả về lỗi cụ thể nhất cho UX tốt.
     */
    fun getPasswordError(password: String): String? {
        return when {
            password.isBlank() -> null // Chưa nhập
            password.length < 6 -> "Mật khẩu phải có ít nhất 6 ký tự"
            !password.any { it.isUpperCase() } -> "Cần ít nhất 1 chữ cái viết hoa"
            !password.any { it.isDigit() } -> "Cần ít nhất 1 chữ số"
            else -> null
        }
    }
}
```

```
}  
}  
  
/**  
 * Lấy thông báo lỗi confirm password (null = hợp lệ).  
 */  
fun getConfirmPasswordError(password: String, confirmPassword: String): String? {  
    return when {  
        confirmPassword.isBlank() -> null // Chưa nhập  
        confirmPassword != password -> "Mật khẩu không khớp"  
        else -> null  
    }  
}  
}
```

File: auth/viewmodel/LoginViewModel.kt

```

package com.team.smartnutrition.auth.viewmodel

import android.app.Activity
import androidx.credentials.CredentialManager
import androidx.credentials.GetCredentialRequest
import androidx.credentials.GetCredentialResponse
import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import com.google.android.libraries.identity.googleid.GetGoogleIdOption
import com.google.android.libraries.identity.googleid.GoogleIdTokenCredential
import com.google.firebase.auth.GoogleAuthProvider
import com.team.smartnutrition.auth.data.UserRepository
import com.team.smartnutrition.auth.util.Validators
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.flow.update
import kotlinx.coroutines.launch

/**
 * =====
 * LOGIN VIEW MODEL
 * =====
 *
 * Xử lý:
 * - Đăng nhập Email/Password
 * - Đăng nhập Google (Credential Manager)
 * - Kiểm tra user đã có profile chưa → navigate phù hợp
 */

data class LoginUiState(
    val email: String = "",
    val password: String = "",
    val isLoading: Boolean = false,
    val errorMessage: String? = null,
    val navigateTo: LoginDestination? = null
)

enum class LoginDestination {
    HOME, // Đã có profile → vào Home
    PROFILE_SETUP // Chưa có profile → setup lần đầu
}

class LoginViewModel : ViewModel() {

    private val repository = UserRepository()

    private val _uiState = MutableStateFlow(LoginUiState())
    val uiState: StateFlow<LoginUiState> = _uiState.asStateFlow()

    fun updateEmail(email: String) {
        _uiState.update { it.copy(email = email, errorMessage = null) }
    }

    fun updatePassword(password: String) {
        _uiState.update { it.copy(password = password, errorMessage = null) }
    }

    /** Kiểm tra form hợp lệ (dùng cho enable/disable button) */
    fun isValidForm(): Boolean {
        val state = _uiState.value
        return Validators.isValidEmail(state.email) && state.password.isNotBlank()
    }

```

```

    }

    /**
     * Đăng nhập bằng Email/Password.
     */
    fun signInWithEmail() {
        val state = _uiState.value
        if (!Validators.isValidEmail(state.email)) {
            _uiState.update { it.copy(errorMessage = "Email không hợp lệ") }
            return
        }

        viewModelScope.launch {
            _uiState.update { it.copy(isLoading = true, errorMessage = null) }
            try {
                val user = repository.signInWithEmail(state.email, state.password)
                checkProfileAndNavigate(user.uid)
            } catch (e: Exception) {
                _uiState.update {
                    it.copy(
                        isLoading = false,
                        errorMessage = mapFirebaseError(e.message)
                    )
                }
            }
        }
    }

    /**
     * Đăng nhập bằng Google Credential Manager.
     * Cần Activity context để hiện dialog chọn tài khoản.
     */
    fun signInWithGoogle(activity: Activity) {
        viewModelScope.launch {
            _uiState.update { it.copy(isLoading = true, errorMessage = null) }
            try {
                // 1. Tạo Google ID option
                val googleIdOption = GetGoogleIdOption.Builder()
                    .setFilterByAuthorizedAccounts(false)
                    .setServerClientId(getWebClientId(activity))
                    .build()

                // 2. Tạo credential request
                val request = GetCredentialRequest.Builder()
                    .addCredentialOption(googleIdOption)
                    .build()

                // 3. Gọi Credential Manager
                val credentialManager = CredentialManager.create(activity)
                val result: GetCredentialResponse =
                    credentialManager.getCredential(activity, request)

                // 4. Lấy Google ID Token
                val googleIdTokenCredential =
                    GoogleIdTokenCredential.createFrom(result.credential.data)
                val idToken = googleIdTokenCredential.idToken

                // 5. Đăng nhập Firebase bằng credential
                val firebaseCredential = GoogleAuthProvider.getCredential(idToken, null)
                val user = repository.signInWithCredential(firebaseCredential)

                checkProfileAndNavigate(user.uid)
            }
        }
    }

```

```

        } catch (e: Exception) {
            _uiState.update {
                it.copy(
                    isLoading = false,
                    errorMessage = "Đăng nhập Google thất bại: ${e.localizedMessage}"
                )
            }
        }
    }
}

/**
 * Kiểm tra user đã có profile trên Firestore chưa.
 * - Có → navigate HOME
 * - Chưa → navigate PROFILE_SETUP
 */
private suspend fun checkProfileAndNavigate(uid: String) {
    try {
        val hasProfile = repository.hasProfile(uid)
        _uiState.update {
            it.copy(
                isLoading = false,
                navigateTo = if (hasProfile) LoginDestination.HOME
                else LoginDestination.PROFILE_SETUP
            )
        }
    } catch (e: Exception) {
        // Firestore offline → default navigate to HOME
        _uiState.update {
            it.copy(isLoading = false, navigateTo = LoginDestination.HOME)
        }
    }
}

/** Reset navigation state sau khi đã navigate */
fun onNavigated() {
    _uiState.update { it.copy(navigateTo = null) }
}

/** Clear error message */
fun clearError() {
    _uiState.update { it.copy(errorMessage = null) }
}

/**
 * Lấy Web Client ID từ google-services.json (qua strings.xml).
 * Firebase tự generate string resource "default_web_client_id".
 */
private fun getWebClientId(activity: Activity): String {
    // 1. Thử lấy trực tiếp thông qua R class của package chính (com.team.smartnutrition)
    // Cách này an toàn nhất vì tránh được lỗi khi chạy debug build có applicationIdSuffix
    try {
        val resId = com.team.smartnutrition.R.string.default_web_client_id
        val clientId = activity.getString(resId)
        if (clientId.isNotEmpty()) {
            return clientId
        }
    } catch (e: Throwable) {
        // Bỏ qua nếu có lỗi biên dịch hoặc runtime
    }

    // 2. Thử tìm qua Identifier với package name cứng của ứng dụng

```

```

        val hardcodedPackageResId = activity.resources.getIdentifier(
            "default_web_client_id", "string", "com.team.smartnutrition"
        )
        if (hardcodedPackageResId != 0) {
            return activity.getString(hardcodedPackageResId)
        }

        // 3. Thử tìm qua Identifier với activity.packageName (cách cũ)
        val activityPackageResId = activity.resources.getIdentifier(
            "default_web_client_id", "string", activity.packageName
        )
        if (activityPackageResId != 0) {
            return activity.getString(activityPackageResId)
        }

        // 4. Nếu tất cả đều thất bại, fallback về Web Client ID lấy từ google-services.json hiện tại
        return "403911937731-p004v6umhf2fgertf1mscq325cdprsg9.apps.googleusercontent.com"
    }

    /** Map Firebase error codes sang tiếng Việt */
    private fun mapFirebaseError(message: String?): String {
        return when {
            message == null -> "Đã xảy ra lỗi"
            message.contains("timeout") || message.contains("timed out") || message.contains("Timed out") ->
                "Quá thời gian kết nối. Vui lòng kiểm tra mạng hoặc đảm bảo bạn đã cấu hình vân tay SHA-1/SHA-256 trong Firebase Console."
            message.contains("Configuration not found") || message.contains("CONFIGURATION_NOT_FOUND") || message.contains("configuration not found") ->
                "Lỗi cấu hình Firebase: Bạn chưa bật phương thức đăng nhập Email/Password trong Firebase Console (Authentication > Sign-in method)"
            message.contains("INVALID_LOGIN_CREDENTIALS") ||
            message.contains("invalid-credential") -> "Email hoặc mật khẩu không đúng"
            message.contains("user-not-found") -> "Tài khoản không tồn tại"
            message.contains("wrong-password") -> "Mật khẩu không đúng"
            message.contains("too-many-requests") -> "Quá nhiều lần thử. Vui lòng thử lại sau"
            message.contains("network") -> "Lỗi mạng. Kiểm tra kết nối Internet"
            message.contains("user-disabled") -> "Tài khoản đã bị vô hiệu hóa"
            else -> "Đăng nhập thất bại: $message"
        }
    }
}

```

File: auth/viewmodel/ProfileSetupViewModel.kt

```

package com.team.smartnutrition.auth.viewmodel

import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import com.google.firebase.auth.FirebaseAuth
import com.team.smartnutrition.auth.data.UserRepository
import com.team.smartnutrition.auth.model.User
import com.team.smartnutrition.auth.util.HealthCalculator
import com.team.smartnutrition.auth.util.HealthMetrics
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.flow.update
import kotlinx.coroutines.launch

/**
 * =====
 * PROFILE SETUP VIEW MODEL
 * =====
 *
 * Quản lý multi-step wizard 3 bước:
 * Step 1: Giới tính + Năm sinh + Tên hiển thị
 * Step 2: Chiều cao + Cân nặng
 * Step 3: Mục tiêu + Mức độ vận động → tính toán → lưu
 */

data class ProfileSetupUiState(
    val currentStep: Int = 1, // 1, 2, 3
    val totalSteps: Int = 3,
    // Step 1
    val displayName: String = "",
    val gender: String = "male", // "male" | "female"
    val birthYear: Int = 2003,
    // Step 2
    val heightCm: Int = 170,
    val weightKg: Double = 65.0,
    // Step 3
    val goal: String = "maintain", // "lose_weight" | "maintain" | "gain_muscle"
    val activityLevel: Double = 1.55,
    // Calculated metrics (tính khi chuyển sang step 3)
    val metrics: HealthMetrics? = null,
    // UI state
    val isSaving: Boolean = false,
    val isSaved: Boolean = false, // true → navigate Home
    val errorMessage: String? = null
)

/**
 * Mức độ vận động với label tiếng Việt.
 */
data class ActivityLevelOption(
    val value: Double,
    val label: String,
    val emoji: String
)

val activityLevelOptions = listOf(
    ActivityLevelOption(1.2, "Ít vận động", "🐢"),
    ActivityLevelOption(1.375, "Vận động nhẹ", "🚶"),
    ActivityLevelOption(1.55, "Vận động vừa", "🏃"),
    ActivityLevelOption(1.725, "Vận động nhiều", "🏊"),
    ActivityLevelOption(1.9, "Rất nhiều", "🔥")
)

```



```

)

/**
 * Mục tiêu sức khỏe với label.
 */
data class GoalOption(
    val value: String,
    val label: String,
    val emoji: String,
    val description: String
)

val goalOptions = listOf(
    GoalOption("lose_weight", "Giảm mỡ", "🔥", "TDEE - 500 kcal/ngày"),
    GoalOption("gain_muscle", "Tăng cơ", "💪", "TDEE + 300 kcal/ngày"),
    GoalOption("maintain", "Duy trì", "👉", "Giữ nguyên TDEE")
)

class ProfileSetupViewModel : ViewModel() {

    private val repository = UserRepository()

    private val _uiState = MutableStateFlow(ProfileSetupUiState())
    val uiState: StateFlow<ProfileSetupUiState> = _uiState.asStateFlow()

    // == Step 1 updates ==

    fun updateDisplayName(name: String) {
        _uiState.update { it.copy(displayName = name) }
    }

    fun updateGender(gender: String) {
        _uiState.update { it.copy(gender = gender) }
    }

    fun updateBirthYear(year: Int) {
        _uiState.update { it.copy(birthYear = year) }
    }

    // == Step 2 updates ==

    fun updateHeightCm(height: Int) {
        _uiState.update { it.copy(heightCm = height) }
    }

    fun updateWeightKg(weight: Double) {
        _uiState.update { it.copy(weightKg = weight) }
    }

    // == Step 3 updates ==

    fun updateGoal(goal: String) {
        _uiState.update { it.copy(goal = goal) }
        recalculateMetrics()
    }

    fun updateActivityLevel(level: Double) {
        _uiState.update { it.copy(activityLevel = level) }
        recalculateMetrics()
    }

    // == Navigation ==

```

```

fun nextStep() {
    val state = _uiState.value
    if (state.currentStep < state.totalSteps) {
        val nextStep = state.currentStep + 1
        _uiState.update { it.copy(currentStep = nextStep) }
        // Tính metrics khi chuyển sang step 3
        if (nextStep == 3) {
            recalculateMetrics()
        }
    }
}

fun previousStep() {
    val state = _uiState.value
    if (state.currentStep > 1) {
        _uiState.update { it.copy(currentStep = state.currentStep - 1) }
    }
}

/** Kiểm tra step hiện tại đã điền đủ chưa */
fun isCurrentStepValid(): Boolean {
    val state = _uiState.value
    return when (state.currentStep) {
        1 -> state.displayName.isNotBlank() && state.birthYear in 1940..2015
        2 -> state.heightCm in 100..250 && state.weightKg in 30.0..300.0
        3 -> state.goal.isNotBlank() && state.activityLevel > 0
        else -> false
    }
}

// == Calculate ==

private fun recalculateMetrics() {
    val state = _uiState.value
    val metrics = HealthCalculator.calculateAllMetrics(
        weightKg = state.weightKg,
        heightCm = state.heightCm,
        birthYear = state.birthYear,
        gender = state.gender,
        activityLevel = state.activityLevel,
        goal = state.goal
    )
    _uiState.update { it.copy(metrics = metrics) }
}

// == Save ==

/**
 * Lưu profile lên Firestore.
 * Gọi khi user bấm "Hoàn tất" ở step 3.
 */
fun saveProfile() {
    val currentUser = FirebaseAuth.getInstance().currentUser ?: return
    val state = _uiState.value
    val metrics = state.metrics ?: return

    viewModelScope.launch {
        _uiState.update { it.copy(isSaving = true, errorMessage = null) }
        try {
            val user = User(
                uid = currentUser.uid,

```

```

        email = currentUser.email ?: "",
        displayName = state.displayName,
        gender = state.gender,
        birthYear = state.birthYear,
        heightCm = state.heightCm,
        weightKg = state.weightKg,
        activityLevel = state.activityLevel,
        goal = state.goal,
        bmi = metrics.bmi,
        bmr = metrics.bmr,
        tdee = metrics.tdee,
        proteinTarget = metrics.proteinTarget,
        carbTarget = metrics.carbTarget,
        fatTarget = metrics.fatTarget,
        calorieTarget = metrics.calorieTarget
    )
    repository.saveUser(user)
    _uiState.update { it.copy(isSaving = false, isSaved = true) }
} catch (e: Exception) {
    _uiState.update {
        it.copy(
            isSaving = false,
            errorMessage = "Lưu thất bại: ${e.localizedMessage}"
        )
    }
}
}
}
}

fun onNavigated() {
    _uiState.update { it.copy(isSaved = false) }
}
}
}

```

File: auth/viewmodel/ProfileViewViewModel.kt

```

package com.team.smartnutrition.auth.viewmodel

import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import com.google.firebase.auth.FirebaseAuth
import com.team.smartnutrition.auth.data.UserRepository
import com.team.smartnutrition.auth.model.User
import com.team.smartnutrition.auth.util.HealthCalculator
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.flow.update
import kotlinx.coroutines.launch

/**
 * =====
 * PROFILE VIEW VIEW MODEL
 * =====
 *
 * Xử lý:
 * - Load user profile từ Firestore
 * - Chế độ Edit (cập nhật + recalculate metrics)
 * - Sign Out
 */

data class ProfileViewState(
    val user: User? = null,
    val isLoading: Boolean = true,
    val isEditing: Boolean = false,
    val isSaving: Boolean = false,
    val errorMessage: String? = null,
    val isSignedOut: Boolean = false,
    // Edit fields (chỉ dùng khi isEditing = true)
    val editDisplayName: String = "",
    val editHeightCm: Int = 170,
    val editWeightKg: Double = 65.0,
    val editGender: String = "male",
    val editBirthYear: Int = 2000,
    val editGoal: String = "maintain",
    val editActivityLevel: Double = 1.55
)

class ProfileViewViewModel : ViewModel() {

    private val repository = UserRepository()

    private val _uiState = MutableStateFlow(ProfileViewState())
    val uiState: StateFlow<ProfileViewState> = _uiState.asStateFlow()

    init {
        loadProfile()
    }

    /**
     * Load user profile từ Firestore.
     */
    fun loadProfile() {
        val uid = FirebaseAuth.getInstance().currentUser?.uid ?: return

        viewModelScope.launch {
            _uiState.update { it.copy(isLoading = true, errorMessage = null) }
            try {

```

```

        val user = repository.getUser(uid)
        _uiState.update {
            it.copy(
                user = user,
                isLoading = false,
                errorMessage = if (user == null) "Không tìm thấy hồ sơ" else null
            )
        }
    } catch (e: Exception) {
        _uiState.update {
            it.copy(
                isLoading = false,
                errorMessage = "Lỗi tải hồ sơ: ${e.localizedMessage}"
            )
        }
    }
}

}

}

/**
 * Bắt đầu chế độ chỉnh sửa.
 * Copy dữ liệu hiện tại vào edit fields.
 */
fun startEditing() {
    val user = _uiState.value.user ?: return
    _uiState.update {
        it.copy(
            isEditing = true,
            editDisplayName = user.displayName,
            editHeightCm = user.heightCm,
            editWeightKg = user.weightKg,
            editGender = user.gender,
            editBirthYear = user.birthYear,
            editGoal = user.goal,
            editActivityLevel = user.activityLevel
        )
    }
}

/** Hủy chỉnh sửa */
fun cancelEditing() {
    _uiState.update { it.copy(isEditing = false) }
}

// == Edit field updates ==

fun updateEditDisplayName(name: String) {
    _uiState.update { it.copy(editDisplayName = name) }
}

fun updateEditHeightCm(height: Int) {
    _uiState.update { it.copy(editHeightCm = height) }
}

fun updateEditWeightKg(weight: Double) {
    _uiState.update { it.copy(editWeightKg = weight) }
}

fun updateEditGender(gender: String) {
    _uiState.update { it.copy(editGender = gender) }
}

```

```

fun updateEditBirthYear(year: Int) {
    _uiState.update { it.copy(editBirthYear = year) }
}

fun updateEditGoal(goal: String) {
    _uiState.update { it.copy(editGoal = goal) }
}

fun updateEditActivityLevel(level: Double) {
    _uiState.update { it.copy(editActivityLevel = level) }
}

/**
 * Lưu thay đổi: recalculate metrics + update Firestore.
 */
fun saveChanges() {
    val uid = FirebaseAuth.getInstance().currentUser?.uid ?: return
    val state = _uiState.value
    val currentUser = state.user ?: return

    viewModelScope.launch {
        _uiState.update { it.copy(isSaving = true, errorMessage = null) }
        try {
            // Tính lại tất cả metrics
            val metrics = HealthCalculator.calculateAllMetrics(
                weightKg = state.editWeightKg,
                heightCm = state.editHeightCm,
                birthYear = state.editBirthYear,
                gender = state.editGender,
                activityLevel = state.editActivityLevel,
                goal = state.editGoal
            )

            val updatedUser = currentUser.copy(
                displayName = state.editDisplayName,
                heightCm = state.editHeightCm,
                weightKg = state.editWeightKg,
                gender = state.editGender,
                birthYear = state.editBirthYear,
                goal = state.editGoal,
                activityLevel = state.editActivityLevel,
                bmi = metrics.bmi,
                bmr = metrics.bmr,
                tdee = metrics.tdee,
                proteinTarget = metrics.proteinTarget,
                carbTarget = metrics.carbTarget,
                fatTarget = metrics.fatTarget,
                calorieTarget = metrics.calorieTarget
            )

            repository.saveUser(updatedUser)
            _uiState.update {
                it.copy(
                    user = updatedUser,
                    isEditing = false,
                    isSaving = false
                )
            }
        } catch (e: Exception) {
            _uiState.update {
                it.copy(
                    isSaving = false,

```

```
                errorMessage = "Lưu thất bại: ${e.localizedMessage}"
            )
        }
    }
}

/**
 * Đăng xuất.
 */
fun signOut() {
    repository.signOut()
    _uiState.update { it.copy(isSignedOut = true) }
}

fun onSignedOut() {
    _uiState.update { it.copy(isSignedOut = false) }
}
}
```

File: auth/viewmodel/RegisterViewModel.kt

```

package com.team.smartnutrition.auth.viewmodel

import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import com.team.smartnutrition.auth.data.UserRepository
import com.team.smartnutrition.auth.util.Validators
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.flow.update
import kotlinx.coroutines.launch

/**
 * =====
 * REGISTER VIEW MODEL
 * =====
 *
 * Xử lý đăng ký tài khoản mới.
 * Validation realtime + Firebase createUserWithEmailAndPassword.
 */

data class RegisterUiState(
    val email: String = "",
    val password: String = "",
    val confirmPassword: String = "",
    val isLoading: Boolean = false,
    val errorMessage: String? = null,
    val isRegistered: Boolean = false, // true → navigate ProfileSetup
    // Validation errors (null = hợp lệ hoặc chưa nhập)
    val emailError: String? = null,
    val passwordError: String? = null,
    val confirmPasswordError: String? = null
)

class RegisterViewModel : ViewModel() {

    private val repository = UserRepository()

    private val _uiState = MutableStateFlow(RegisterUiState())
    val uiState: StateFlow<RegisterUiState> = _uiState.asStateFlow()

    fun updateEmail(email: String) {
        _uiState.update {
            it.copy(
                email = email,
                emailError = Validators.getEmailError(email),
                errorMessage = null
            )
        }
    }

    fun updatePassword(password: String) {
        _uiState.update {
            it.copy(
                password = password,
                passwordError = Validators.getPasswordError(password),
                // Re-validate confirm nếu đã nhập
                confirmPasswordError = if (it.confirmPassword.isNotBlank())
                    Validators.getConfirmPasswordError(password, it.confirmPassword)
                else null,
                errorMessage = null
            )
        }
    }

```



```

    }
}

fun updateConfirmPassword(confirmPassword: String) {
    _uiState.update {
        it.copy(
            confirmPassword = confirmPassword,
            confirmPasswordError = Validators.getConfirmPasswordError(
                it.password, confirmPassword
            ),
            errorMessage = null
        )
    }
}

/** Kiểm tra form hợp lệ hoàn toàn */
fun isValid(): Boolean {
    val state = _uiState.value
    return Validators.isValidEmail(state.email) &&
        Validators.isValidPassword(state.password) &&
        Validators.doPasswordsMatch(state.password, state.confirmPassword)
}

/**
 * Đăng ký tài khoản mới.
 */
fun register() {
    val state = _uiState.value

    // Validate lần cuối trước khi gửi
    if (!isValid()) {
        _uiState.update {
            it.copy(
                emailError = Validators.getEmailError(state.email),
                passwordError = Validators.getPasswordError(state.password),
                confirmPasswordError = Validators.getConfirmPasswordError(
                    state.password, state.confirmPassword
                )
            )
        }
    }
    return
}

viewModelScope.launch {
    _uiState.update { it.copy(isLoading = true, errorMessage = null) }
    try {
        repository.registerWithEmail(state.email, state.password)
        _uiState.update { it.copy(isLoading = false, isRegistered = true) }
    } catch (e: Exception) {
        _uiState.update {
            it.copy(
                isLoading = false,
                errorMessage = mapRegisterError(e.message)
            )
        }
    }
}

/** Reset navigation state */
fun onNavigated() {
    _uiState.update { it.copy(isRegistered = false) }
}

```

```
}

/** Map Firebase register errors sang tiếng Việt */
private fun mapRegisterError(message: String?): String {
    return when {
        message == null -> "Đã xảy ra lỗi"
        message.contains("timeout") || message.contains("timed out") || message.contains("Timed out") ->
            "Quá thời gian kết nối. Vui lòng kiểm tra mạng hoặc đảm bảo bạn đã cấu hình vân tay SHA-1/SHA-256 trong Firebase Console."
        message.contains("Configuration not found") || message.contains("CONFIGURATION_NOT_FOUND") || message.contains("configuration not found") ->
            "Lỗi cấu hình Firebase: Bạn chưa bật phương thức đăng nhập Email/Password trong Firebase Console (Authentication > Sign-in method)"
        message.contains("email-already-in-use") -> "Email đã được sử dụng"
        message.contains("weak-password") -> "Mật khẩu quá yếu"
        message.contains("invalid-email") -> "Email không hợp lệ"
        message.contains("network") -> "Lỗi mạng. Kiểm tra kết nối Internet"
        else -> "Đăng ký thất bại: $message"
    }
}
}
```

File: auth/viewmodel/WeightLogViewModel.kt

```

package com.team.smartnutrition.auth.viewmodel

import androidx.lifecycle.ViewModel
import androidx.lifecycle.viewModelScope
import com.google.firebase.auth.FirebaseAuth
import com.team.smartnutrition.auth.data.UserRepository
import com.team.smartnutrition.auth.model.WeightEntry
import com.team.smartnutrition.auth.util.HealthCalculator
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.flow.update
import kotlinx.coroutines.launch
import java.time.LocalDate
import java.time.format.DateTimeFormatter

/**
 * =====
 * WEIGHT LOG VIEW MODEL
 * =====
 *
 * Xử lý:
 * - Nhập cân nặng hôm nay
 * - Tính lại BMI realtime
 * - Lưu vào weightLog/{date} + update user profile
 * - Load lịch sử cân nặng
 */

data class WeightLogUiState(
    val currentWeight: String = "", // String để user nhập linh hoạt
    val heightCm: Int = 170, // Lấy từ user profile để tính BMI
    val calculatedBmi: Double? = null, // BMI mới (tính realtime khi nhập)
    val history: List<WeightEntry> = emptyList(),
    val isLoading: Boolean = true,
    val isSaving: Boolean = false,
    val saveSuccess: Boolean = false,
    val errorMessage: String? = null
)

class WeightLogViewModel : ViewModel() {

    private val repository = UserRepository()
    private val dateFormatter = DateTimeFormatter.ofPattern("yyyy-MM-dd")

    private val _uiState = MutableStateFlow(WeightLogUiState())
    val uiState: StateFlow<WeightLogUiState> = _uiState.asStateFlow()

    init {
        loadData()
    }

    /**
     * Load user height (để tính BMI) + lịch sử cân nặng.
     */
    fun loadData() {
        val uid = FirebaseAuth.getInstance().currentUser?.uid ?: return

        viewModelScope.launch {
            _uiState.update { it.copy(isLoading = true, errorMessage = null) }
            try {
                // Load user profile để lấy heightCm
                val user = repository.getUser(uid)
            }
        }
    }

```

```

        val heightCm = user?.heightCm ?: 170

        // Load lịch sử
        val history = repository.getWeightHistory(uid)

        _uiState.update {
            it.copy(
                heightCm = heightCm,
                history = history,
                isLoading = false,
                // Pre-fill cân nặng hiện tại nếu có
                currentWeight = user?.weightKg?.toString() ?: ""
            )
        }
    } catch (e: Exception) {
        _uiState.update {
            it.copy(
                isLoading = false,
                errorMessage = "Lỗi tải dữ liệu: ${e.localizedMessage}"
            )
        }
    }
}

/**
 * Cập nhật cân nặng nhập vào.
 * Tính BMI mới realtime.
 */
fun updateWeight(weight: String) {
    val weightKg = weight.toDoubleOrNull()
    val bmi = if (weightKg != null && weightKg > 0) {
        HealthCalculator.calculateBmi(weightKg, _uiState.value.heightCm)
    } else null

    _uiState.update {
        it.copy(
            currentWeight = weight,
            calculatedBmi = bmi,
            saveSuccess = false
        )
    }
}

/**
 * Lưu cân nặng hôm nay.
 * - Upsert vào weightLog/{today}
 * - Update user profile weightKg + bmi
 */
fun saveWeight() {
    val uid = FirebaseAuth.getInstance().currentUser?.uid ?: return
    val state = _uiState.value
    val weightKg = state.currentWeight.toDoubleOrNull() ?: return
    val bmi = state.calculatedBmi ?: return

    if (weightKg < 30 || weightKg > 300) {
        _uiState.update { it.copy(errorMessage = "Cân nặng phải từ 30 đến 300 kg") }
        return
    }

    val today = LocalDate.now().format(dateFormatter)

```

```
viewModelScope.launch {
    _uiState.update { it.copy(isSaving = true, errorMessage = null) }
    try {
        repository.logWeight(uid, today, weightKg, bmi)
        // Reload lịch sử
        val history = repository.getWeightHistory(uid)
        _uiState.update {
            it.copy(
                isSaving = false,
                saveSuccess = true,
                history = history
            )
        }
    } catch (e: Exception) {
        _uiState.update {
            it.copy(
                isSaving = false,
                errorMessage = "Lưu thất bại: ${e.localizedMessage}"
            )
        }
    }
}

fun clearError() {
    _uiState.update { it.copy(errorMessage = null) }
}

fun clearSaveSuccess() {
    _uiState.update { it.copy(saveSuccess = false) }
}
}
```